

claude-windows / 188dd06f-9c99-487c-aa7d-b0bee83c629c

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\188dd06f-9c99-487c-aa7d-b0bee83c629c\subagents\workflows\wf\_d9866476-800\agent-ab6ba611cbff9e4db.jsonl`
- SHA-256: `23dad55f52e2bfcabb8baf0b409309f7461b51eed58f32d72dd59a5ae05b6add`
- Source modified: `2026-06-22T09:10:24+00:00`
- Imported at: `2026-07-05T16:48:28+00:00`
- project: `wf\_d9866476-800`
- session_id: `188dd06f-9c99-487c-aa7d-b0bee83c629c`

Transcript

1. user (2026-06-22T09:08:03.303Z)

You are mapping the badminton-highlight-indexer repo, checked out at origin/master under C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/nightly-e2e. Read files under that ABSOLUTE path (Read/Grep/Glob). Be concrete: cite file:line / file::symbol. The goal is to design a NIGHTLY regression that runs the CONFIGURED + CHECKPOINTED pipeline on 2-3 short golden videos and asserts the NUMBER OF REELS (highlight clips) does not change. Return raw structured data, not prose for a human.

DETERMINISM: is the COUNT of reels/segments deterministic for a FIXED (video, config, weights)? Inspect WASB inference (backend/pipeline/detectors/wasb_infer.py, tracknet_runner.py), windowing (backend/eval/windowing.py + trajectory_to_action_windows), the segmenter (TrajectoryHybridSegmenter), and stitching for ANY nondeterminism: RNG without seed, time/date in output, dict-ordering, ffmpeg keyframe variance, float threshold ties, GPU nondeterminism, parallelism. State whether the reel COUNT (not timings/bytes) is safe to assert exactly, and if not, what tolerance is needed. Distinguish count-stability from byte/timing-stability.

2. assistant (2026-06-22T09:08:05.325Z)

I'll map this repository and analyze determinism for the nightly regression testing. Let me start by exploring the structure and understanding the pipeline.

3. user (2026-06-22T09:08:06.542Z)

```
C:\Users\avidu\Projects\badminton-highlight-indexer\claude\worktrees\nightly-e2e\backend\eval\export_wasb_segments.py
C:\Users\avidu\Projects\badminton-highlight-indexer\claude\worktrees\nightly-e2e\backend\eval\segmentation_metrics.py
C:\Users\avidu\Projects\badminton-highlight-indexer\claude\worktrees\nightly-e2e\backend\eval>windowing.py
C:\Users\avidu\Projects\badminton-highlight-indexer\claude\worktrees\nightly-e2e\backend\pipeline\audio\hit_detector.py
C:\Users\avidu\Projects\badminton-highlight-indexer\claude\worktrees\nightly-e2e\backend\pipeline\audio\__init__.py
C:\Users\avidu\Projects\badminton-highlight-indexer\claude\worktrees\nightly-e2e\backend\pipeline\detectors\audio_features.py
C:\Users\avidu\Projects\badminton-highlight-indexer\claude\worktrees\nightly-e2e\backend\pipeline\detectors\base.py
C:\Users\avidu\Projects\badminton-highlight-indexer\claude\worktrees\nightly-e2e\backend\pipeline\detectors\court_gate.py
C:\Users\avidu\Projects\badminton-highlight-indexer\claude\worktrees\nightly-e2e\backend\pipeline\detectors\device.py
C:\Users\avidu\Projects\badminton-highlight-indexer\claude\worktrees\nightly-e2e\backend\pipeline\detectors\fusion_features.py
C:\Users\avidu\Projects\badminton-highlight-indexer\claude\worktrees\nightly-e2e\backend\pipeline\detectors\native_wasb_runner.py
```

```
C:\Users\avidu\Projects\badminton-highlight-  
indexer\.claude\worktrees\nightly-e2e/backend/pipeline/detectors/person_detector.py  
C:\Users\avidu\Projects\badminton-highlight-  
indexer\.claude\worktrees\nightly-e2e/backend/pipeline/detectors/rally_gate.py  
C:\Users\avidu\Projects\badminton-highlight-  
indexer\.claude\worktrees\nightly-e2e/backend/pipeline/detectors/stub_runner.py  
C:\Users\avidu\Projects\badminton-highlight-  
indexer\.claude\worktrees\nightly-e2e/backend/pipeline/detectors/tracknet_runner.py  
C:\Users\avidu\Projects\badminton-highlight-  
indexer\.claude\worktrees\nightly-e2e/backend/pipeline/detectors/wasb_infer.py  
C:\Users\avidu\Projects\badminton-highlight-  
indexer\.claude\worktrees\nightly-e2e/backend/pipeline/detectors/wasb_runner.py  
C:\Users\avidu\Projects\badminton-highlight-  
indexer\.claude\worktrees\nightly-e2e/backend/pipeline/detectors/__init__.py  
C:\Users\avidu\Projects\badminton-highlight-  
indexer\.claude\worktrees\nightly-e2e/backend/pipeline/personalization/serving.py  
C:\Users\avidu\Projects\badminton-highlight-  
indexer\.claude\worktrees\nightly-e2e/backend/pipeline/personalization/__init__.py
```

4. assistant (2026-06-22T09:08:08.758Z)

Let me get the full directory structure and examine key files:

5. user (2026-06-22T09:08:10.005Z)

```
total 207  
drwxr-xr-x 1 avidu 197609    0 Jun 22 14:36 .  
drwxr-xr-x 1 avidu 197609    0 Jun 22 14:36 ..  
-rw-r--r-- 1 avidu 197609  611 Jun 22 14:36 .coveragerc  
-rw-r--r-- 1 avidu 197609   87 Jun 22 14:36 .git  
-rw-r--r-- 1 avidu 197609  947 Jun 22 14:36 .gitattributes  
drwxr-xr-x 1 avidu 197609    0 Jun 22 14:36 .github  
-rw-r--r-- 1 avidu 197609 1117 Jun 22 14:36 .gitignore  
-rw-r--r-- 1 avidu 197609  5525 Jun 22 14:36 CLAUDE.md  
-rw-r--r-- 1 avidu 197609 35347 Jun 22 14:36 README.md  
drwxr-xr-x 1 avidu 197609    0 Jun 22 14:36 backend  
-rw-r--r-- 1 avidu 197609  2735 Jun 22 14:36 config.json  
drwxr-xr-x 1 avidu 197609    0 Jun 22 14:36 deploy  
drwxr-xr-x 1 avidu 197609    0 Jun 22 14:36 docs  
drwxr-xr-x 1 avidu 197609    0 Jun 22 14:36 eval_baselines  
-rw-r--r-- 1 avidu 197609  2344 Jun 22 14:36 mypy.ini  
-rw-r--r-- 1 avidu 197609  4525 Jun 22 14:36 pyproject.toml  
-rw-r--r-- 1 avidu 197609   245 Jun 22 14:36 pytest.ini  
-rw-r--r-- 1 avidu 197609   642 Jun 22 14:36 requirements.txt  
-rw-r--r-- 1 avidu 197609   884 Jun 22 14:36 ruff.toml  
-rw-r-xr-x 1 avidu 197609  2055 Jun 22 14:36 run_all_tests.py  
-rw-r--r-- 1 avidu 197609  5620 Jun 22 14:36 run_eval.py  
-rw-r--r-- 1 avidu 197609  4356 Jun 22 14:36 run_regression.py  
drwxr-xr-x 1 avidu 197609    0 Jun 22 14:36 scratch  
drwxr-xr-x 1 avidu 197609    0 Jun 22 14:36 scripts  
drwxr-xr-x 1 avidu 197609    0 Jun 22 14:36 tests  
drwxr-xr-x 1 avidu 197609    0 Jun 22 14:36 tools  
drwxr-xr-x 1 avidu 197609    0 Jun 22 14:36 training
```

6. assistant (2026-06-22T09:08:11.857Z)

Now let me examine the key determinism-critical files in parallel:

7. user (2026-06-22T09:08:13.263Z)

```
1      """WASB shuttle-trajectory inference on an arbitrary frame sequence (runs in WSL).  
2  
3      WASB-SBDT only ships *dataset* evaluation; this is a thin wrapper that reuses its  
4      own building blocks (ImageDataset transform + detector + online tracker) to run on
```

```

5     any folder of extracted frames and emit a trajectory CSV in the exact shape the
6     indexer's `tracknet_runner.parse_trajectory_csv` already consumes:
7
8     Frame,Visibility,X,Y
9
10    It must run INSIDE the WSL `wasb` conda env, from the WASB-SBDT `src/` dir (it
11    imports WASB's `detectors`/`trackers`/`dataloaders`). The Windows-side adapter
12    (`wasb_runner.py`) stages this file + the frames into WSL and invokes it.
13
14    Resumable across reboots
15    -----
16    The GPU detector pass (one forward per sliding window ? ~21k for a 6-min clip) is
17    the slow, expensive part; the CPU tracker pass that turns detections into a
18    trajectory is cheap and *stateful* (it must see every frame in order, so it can't
19    resume mid-stream). We exploit that split:
20
21    * The detector pass writes its per-window raw outputs incrementally to
22    ``<cache>/det_raw.jsonl`` (flushed+fsync'd every ``--flush-every`` windows) and
23    records progress in ``<cache>/manifest.json`` (atomic write). A reboot loses at
24    most ``--flush-every`` windows of GPU work instead of the whole pass.
25    * On restart we replay the cached windows, skip the DataLoader ahead to the first
26    un-cached window, and continue. Once every window is cached, the detector pass
27    is skipped entirely and we just re-run the (cheap, deterministic) tracker over
28    the full ordered cache -> trajectory CSV. So a lost trajectory CSV costs seconds,
29    not the GPU hour.
30
31    The cache is keyed by the output path (``<out>_wasbcache/`` by default), lives next
32    to the output (NOT in %TEMP%), and is invalidated automatically if the frames dir,
33    weights, sport, window size, or frame count change.
34
35    Usage (in WSL, from ~/models/WASB-SBDT/src):
36    python wasb_infer.py --frames_dir /path/frames --weights
37    ../pretrained_weights/wasb_badminton_best.pth.tar \
38    --out /path/traj.csv [--sport badminton] [--limit N] [--cache-dir DIR] [--flush-
39    every 1000] [--fresh]
40    """
41    import os
42    import os.path as osp
43    import csv
44    import glob
45    import json
46    import argparse
47    import logging
48    from collections import defaultdict
49    from contextlib import contextmanager
50
51    logger = logging.getLogger(__name__)
52
53    CACHE_VERSION = 1
54    _DET_FILE = "det_raw.jsonl"
55    _MANIFEST_FILE = "manifest.json"
56
57    def _build_cfg(weights: str, sport: str, device: str = "cuda"):
58        """Compose the WASB eval config via Hydra, overriding model/weights/device.
59
60        ``device`` is ``cuda`` (default ? the historical WSL behaviour, byte-parity
61        preserved),
62        ``mps`` (Apple) or ``cpu``. Only the single-GPU CUDA path requests
63        ``runner.gpus=[0]``;
64        cpu/mps run with no GPU list so the detector places tensors on ``device``.
65        """
66        from hydra import compose, initialize
67        gpus = "[0]" if device == "cuda" else "[]" # single GPU on cuda; none for cpu/mps
68        with initialize(config_path="configs", version_base=None):

```

```

66         cfg = compose(config_name="eval", overrides=[
67             f"dataset={sport}",
68             "model=wasb",
69             f"detector.model_path={weights}",
70             f"runner.device={device}",
71             f"runner.gpus={gpus}", # default config assumes multiple GPUs; pin to this
box
72         ])
73     return cfg
74
75
76     def _frame_id(path: str) -> int:
77         """Best-effort integer frame id from a frame filename (e.g. frame_000180.png ->
180)."""
78         stem = osp.splitext(osp.basename(path))[0]
79         digits = "".join(ch for ch in stem if ch.isdigit())
80         return int(digits) if digits else -1
81
82
83     # ----- #
84     # Resumable detector cache (pure-Python, no torch/numpy ? unit-testable)
85     # ----- #
86     def _cache_paths(cache_dir: str):
87         return osp.join(cache_dir, _DET_FILE), osp.join(cache_dir, _MANIFEST_FILE)
88
89
90     def default_cache_dir(out_csv: str) -> str:
91         """Stable cache dir next to the trajectory output (survives reboots; not %TEMP%)."""
92         d = osp.dirname(osp.abspath(out_csv))
93         stem = osp.splitext(osp.basename(out_csv))[0]
94         return osp.join(d, stem + "_wasbcache")
95
96
97     def _atomic_write_text(path: str, text: str) -> None:
98         """Write then rename so a crash mid-write can't leave a half-written file."""
99         tmp = path + ".tmp"
100         with open(tmp, "w") as f:
101             f.write(text)
102             f.flush()
103             os.fsync(f.fileno())
104         os.replace(tmp, path)
105
106
107     def _det_plain(det) -> list:
108         """A detector candidate dict {'xy': np.array([x,y]), 'score', 'scale'} -> JSON-able
list."""
109         xy = det["xy"]
110         return [float(xy[0]), float(xy[1]), float(det["score"]), int(det["scale"])]
111
112
113     def _dets_to_tracker(plain_list):
114         """Inverse of _det_plain: rebuild the dicts the tracker expects (xy MUST be a numpy
array,
115         the tracker does vector math / np.linalg.norm on it)."""
116         import numpy as np
117         return [{"xy": np.array([p[0], p[1]], dtype=float), "score": p[2], "scale": p[3]}
118                 for p in plain_list]
119
120
121     def write_manifest(cache_dir: str, manifest: dict) -> None:
122         _, mpath = _cache_paths(cache_dir)
123         _atomic_write_text(mpath, json.dumps(manifest, indent=2))
124
125
126     def read_manifest(cache_dir: str):

```

```

127     _, mpath = _cache_paths(cache_dir)
128     if not osp.exists(mpath):
129         return None
130     try:
131         with open(mpath) as f:
132             return json.load(f)
133     except (json.JSONDecodeError, OSError):
134         return None
135
136
137     def manifest_compatible(manifest: dict, *, frames_dir: str, weights: str, sport: str,
138                             frames_in: int, frames_total: int, device: str = "cuda") ->
139     bool:
140         """A cache is reusable only if the run that produced it matches this run's inputs.
141         ``device`` defaults to ``cuda`` so caches written before device-keying (no
142         ``device`` field) stay compatible with a CUDA run ? but a cpu/mps run will NOT reuse a cuda
143         cache (detections can differ numerically across devices), and vice-versa.
144         """
145         if not manifest or manifest.get("version") != CACHE_VERSION:
146             return False
147         return (
148             manifest.get("frames_dir") == osp.abspath(frames_dir)
149             and manifest.get("weights") == weights
150             and manifest.get("sport") == sport
151             and manifest.get("frames_in") == frames_in
152             and manifest.get("frames_total") == frames_total
153             and manifest.get("device", "cuda") == device
154         )
155
156
157     def reset_cache(cache_dir: str) -> None:
158         """Drop any existing cache so the next run starts clean."""
159         det_jsonl, mpath = _cache_paths(cache_dir)
160         for p in (det_jsonl, mpath):
161             if osp.exists(p):
162                 os.remove(p)
163
164
165     def load_and_clean_cache(cache_dir: str):
166         """Read the longest *contiguous* (w == line index) valid prefix of det_raw.jsonl,
167         rebuild the per-frame detection accumulation, and rewrite the file to exactly that
168         prefix so a trailing half-written line from a crash can't corrupt future appends.
169
170         Returns (windows_done, det_by_fid) where det_by_fid maps frame_id -> list of
171         [x, y, score, scale] candidates (accumulated across every window the frame is in,
172         in window order ? identical to a non-resumed run).
173         """
174         det_jsonl, _ = _cache_paths(cache_dir)
175         det_by_fid = defaultdict(list)
176         valid: list = []
177         if osp.exists(det_jsonl):
178             with open(det_jsonl, "r") as f:
179                 for line in f:
180                     s = line.strip()
181                     if not s:
182                         continue
183                     try:
184                         obj = json.loads(s)
185                     except json.JSONDecodeError:
186                         break # truncated trailing line (crash mid-
flush)
187                     if obj.get("w") != len(valid): # non-contiguous -> stop at the gap

```

```

188         break
189         valid.append(s)
190         for fid, plain in obj["f"]:
191             det_by_fid[int(fid)].extend([list(p) for p in plain])
192         # Guarantee a clean append boundary by rewriting only the good prefix.
193         _atomic_write_text(det_jsonl, ("\n".join(valid) + "\n") if valid else "")
194     return len(valid), det_by_fid
195
196
197 def _append_windows(fh, objs) -> None:
198     """Append window records as JSONL and durably flush (bounded loss on crash)."""
199     for o in objs:
200         fh.write(json.dumps(o, separators=(",", ":")) + "\n")
201     fh.flush()
202     os.fsync(fh.fileno())
203
204
205 def _atomic_write_trajectory(out_csv: str, rows) -> None:
206     os.makedirs(osp.dirname(osp.abspath(out_csv)), exist_ok=True)
207     tmp = out_csv + ".tmp"
208     with open(tmp, "w", newline="") as f:
209         w = csv.writer(f)
210         w.writerow(["Frame", "Visibility", "X", "Y"])
211         w.writerows(rows)
212         f.flush()
213     os.fsync(f.fileno())
214     os.replace(tmp, out_csv)
215
216
217 # ----- #
218 # Frame addressing + windowing (pure; no torch/cv2 ? unit-testable)
219 # ----- #
220 _STREAM_FRAME_FMT = "{:08d}.png"
221
222
223 def synthetic_frame_path(index: int) -> str:
224     """Stable, index-encoding frame name used by the streaming path.
225
226     It mirrors the on-disk names ``extract_frames`` writes (``{:08d}.png``) so the
227     downstream integer-id parser (``_frame_id``) yields the SAME frame id whether the
228     frames came from disk or from the in-memory stream. The streaming image loader
229     inverts this name back to an index to fetch the decoded frame.
230     """
231     return _STREAM_FRAME_FMT.format(int(index))
232
233
234 def build_windows(frames: list, frames_in: int):
235     """Sliding windows of ``frames_in`` consecutive frame *paths* (the unit of GPU work
236     + checkpoint). Pure list math, identical for the disk and streaming paths.
237
238     Returns a list of windows, each a list of ``frames_in`` consecutive paths
239     (``[frames[i:i+frames_in] for i in range(len(frames) - frames_in + 1)]``). The
240     caller wraps each window into the ``{"frames", "annos"}`` sample shape WASB's
241     ``ImageDataset`` expects; keeping that out of here stays torch-free for unit tests.
242     """
243     return [frames[i:i + frames_in] for i in range(len(frames) - frames_in + 1)]
244
245
246 class SequentialFrameStore:
247     """In-memory sliding buffer over a forward-only frame reader, sized for the
248     detector's sliding-window access pattern (peak O(window) frames, not O(video)).
249
250     The detector DataLoader runs with ``shuffle=False`` and (in streaming mode)
251     ``num_workers=0``. ``ImageDataset.__getitem__(i)`` reads the frames of window ``i``
252     in order ? ``i, i+1, ..., i+window-1`` ? and samples are requested ``i = start,

```

```

253     start+1, ...``. So the global read sequence dips back by ``window-1`` at each window
254     boundary (``0,1,2, 1,2,3, 2,3,4, ...`` for ``window=3``); the highest index seen so
255     far never decreases. We keep only frames at or above ``max_seen - (window-1)`` (the
256     earliest index any not-yet-finished window can still ask for) and decode each source
257     frame exactly once via the forward-only ``reader(index)``.
258     """
259
260     def __init__(self, reader, total: int, window: int = 1, start_index: int = 0):
261         self._reader = reader
262         self._total = int(total)
263         self._window = max(1, int(window))
264         self._buf: dict = {}
265         # On a resumed run the detector's first needed frame is `start_index`; begin
266         # pulling there so the reader can skip (seek past) the already-cached prefix
267         # instead of decoding 0..start_index-1 just to throw them away.
268         self._next = int(start_index)      # next index not yet pulled from the reader
269         self._max_seen = int(start_index) - 1
270         self._floor = int(start_index)     # lowest index we still keep (never evict
below)
271
272     def get(self, index: int):
273         index = int(index)
274         if index < self._floor:
275             raise KeyError(
276                 f"frame {index} already evicted (below floor {self._floor}); the access
"
277                 f"pattern must stay within a window of the highest frame seen")
278         if index >= self._total:
279             raise KeyError(f"frame {index} out of range (total {self._total})")
280         # Pull forward from the reader until the requested index is buffered.
281         while self._next <= index:
282             self._buf[self._next] = self._reader(self._next)
283             self._next += 1
284         if index > self._max_seen:
285             self._max_seen = index
286         # Evict frames older than the earliest a still-running window could re-request:
287         # once we've seen index `m`, no future window starts before `m - (window-1)`.
288         new_floor = max(self._floor, self._max_seen - (self._window - 1))
289         for k in range(self._floor, new_floor):
290             self._buf.pop(k, None)
291         self._floor = new_floor
292         return self._buf[index]
293
294
295     def _index_from_synthetic_path(path: str) -> int:
296         """Invert ``synthetic_frame_path``: a frame path -> its integer source index.
297
298         Reuses ``_frame_id`` (digits-only parse) so the index and the cached frame-id stay
299         in lockstep with the on-disk naming scheme.
300         """
301         return _frame_id(path)
302
303
304     def _bgr_to_pil_rgb(frame_bgr):
305         """Convert an in-memory BGR uint8 frame (as ``cv2.VideoCapture`` yields) to the
EXACT
306         PIL RGB image WASB's disk path produces.
307
308         The disk path is ``frame_bgr -> cv2.imwrite(PNG) ->
Image.open(PNG).convert('RGB')``;
309         because PNG is lossless that round-trip equals ``cv2.cvtColor(frame_bgr,
COLOR_BGR2RGB)``
310         bit-for-bit (verified empirically, max|diff|=0). Returning that here makes the
streamed
311         frame indistinguishable from the on-disk one to everything downstream.

```

```

312     """
313     import cv2
314     from PIL import Image
315     return Image.fromarray(cv2.cvtColor(frame_bgr, cv2.COLOR_BGR2RGB))
316
317
318     def _make_streamed_read_image(store, real_read_image):
319         """Build the ``read_image`` replacement used during a streaming detector pass.
320
321         A synthetic frame path -> the in-memory decoded frame for its index (as a PIL RGB
322         image,
323         matching ``read_image``'s real return type); any path that doesn't parse to an index
324         falls
325         through to ``real_read_image``. Pure ? imports no WASB module ? so it is unit-
326         testable
327         without the WSL-only ``dataloaders`` package.
328         """
329         def _read_image(path, *args, **kwargs):
330             idx = _index_from_synthetic_path(path)
331             if idx < 0:
332                 return real_read_image(path, *args, **kwargs)
333             return _bgr_to_pil_rgb(store.get(idx))
334         return _read_image
335
336     @contextmanager
337     def _streaming_read_image_patch(frame_loader, total: int, window: int = 1, start_index:
338     int = 0):
339         """Scope a shim over WASB's ``read_image`` so ``ImageDataset`` is fed in-memory
340         decoded
341         frames during the detector pass, byte-for-byte as it would over real PNGs.
342
343         WASB's ``ImageDataset.__getitem__`` reads each frame via ``read_image(path)`` ? NOT
344         ``cv2.imread``. ``read_image`` (``utils.utils``) does
345         ``Image.open(path).convert('RGB')``,
346         returning a PIL **RGB** image, after an ``osp.exists(path)`` guard. We therefore
347         feed it a
348         PIL RGB image built from the streamed BGR frame (see ``_bgr_to_pil_rgb``), which is
349         byte-identical to the disk round-trip, and the shim never touches the filesystem so
350         the
351         ``osp.exists`` guard is sidestepped for synthetic stream paths.
352
353         We patch ``dataloaders.dataset_loader.read_image`` ? the binding the consumer
354         actually
355         calls (``dataset_loader`` does ``from utils import read_image``, so the name lives
356         in that
357         module's namespace; patching ``utils.read_image`` would NOT rebind the already-
358         imported
359         reference).
360
361         ``frame_loader(index) -> BGR uint8 ndarray`` is a forward-only sequential decoder
362         wrapped
363         in a ``SequentialFrameStore`` (sliding buffer sized to ``window`` = ``frames_in``);
364         on a
365         resume we start at ``start_index`` so the decoder seeks past already-cached windows.
366         The
367         original reader is restored on exit.
368         """
369         from dataloaders import dataset_loader as _dl
370         store = SequentialFrameStore(frame_loader, total, window=window,
371         start_index=start_index)
372         real_read_image = _dl.read_image
373         # Rebind read_image in the consuming module for the duration of the detector pass;
374         # restore on exit. (Plain assignment, not setattr-with-constant, to stay ruff
375         B010-clean.)

```



```

361     _dl.read_image = _make_streamed_read_image(store, real_read_image) # type:
ignore[assignment]
362     try:
363         yield store
364     finally:
365         _dl.read_image = real_read_image # type: ignore[assignment]
366
367
368     # ----- #
369     # Inference
370     # ----- #
371     def run(frames, weights: str, out_csv: str, sport: str = "badminton",
372            limit: int = 0, batch_size: int = 8, num_workers: int = 4,
373            log_every_batches: int = 50, cache_dir: "str | None" = None,
374            flush_every: int = 1000, fresh: bool = False,
375            frame_loader=None, source_key: "str | None" = None,
376            device: str = "cuda") -> str:
377         """Run WASB detection+tracking over an ordered frame source -> trajectory CSV.
378
379         Two equivalent ways to supply pixels (output is bit-identical between them):
380
381         * **frames-on-disk** (default): ``frames`` is a directory path string; frames are
382           globbed (``*.png``/``*.jpg``) and ``ImageDataset`` reads each file via
383           ``cv2.imread``.
384         * **streaming** (fast path): ``frames`` is a pre-built ordered list of (synthetic)
385           frame paths and ``frame_loader(index) -> BGR uint8 ndarray`` supplies the decoded
386           pixels in-memory. We scope a shim over WASB's ``read_image`` (the actual per-frame
387           reader, which returns a PIL RGB image) over the DataLoader pass so every other
388           line of
389           ``ImageDataset.__getitem__`` runs UNCHANGED ? the tensor the detector sees is
390           identical
391           to the disk round-trip (``cv2.imwrite`` PNG -> ``Image.open().convert('RGB')`` ==
392           ``cvtColor(bgr, BGR2RGB)``, verified byte-identical). Streaming forces
393           ``num_workers=0``
394           (the in-process shim + buffer can't cross worker processes).
395
396         ``source_key`` overrides the cache-keying identity (defaults to the abspath of the
397         frames dir); the streaming path passes a stable ``video:<abspath>`` key so a resume
398         after reboot still matches.
399         """
400         import time
401         import torch
402         from torch.utils.data import DataLoader
403         from detectors import build_detector
404         from trackers import build_tracker
405         from dataloaders import build_img_transforms, build_seq_transforms
406         from dataloaders.dataset_loader import ImageDataset
407         from utils import Center
408
409         streaming = frame_loader is not None
410
411         cfg = _build_cfg(weights, sport, device)
412         frames_in = int(cfg["model"]["frames_in"])
413         input_wh = (int(cfg["model"]["inp_width"]), int(cfg["model"]["inp_height"]))
414         output_wh = (int(cfg["model"]["out_width"]), int(cfg["model"]["out_height"]))
415
416         if streaming:
417             # `frames` is already the ordered list of (synthetic) frame paths.
418             if not isinstance(frames, (list, tuple)):
419                 raise SystemExit("streaming mode requires `frames` to be a list of frame
paths")
420             frames = list(frames)
421             frames_dir = source_key or "stream"
422         else:
423             frames_dir = frames

```

```

420         frames = sorted(glob.glob(osp.join(frames_dir, "*.png")) +
glob.glob(osp.join(frames_dir, "*.jpg")))
421     if limit:
422         frames = frames[:limit]
423     if len(frames) < frames_in:
424         where = frames_dir if not streaming else "video stream"
425         raise SystemExit(f"need >= {frames_in} frames, found {len(frames)} in {where}")
426
427     # Sliding windows of `frames_in` consecutive frames (the unit of GPU work +
checkpoint).
428     samples = []
429     for window in build_windows(frames, frames_in):
430         annos = [{"frame_path": p, "center": Center(is_visible=False, x=-1.0, y=-1.0)}
for p in window]
431         samples.append({"frames": window, "annos": annos})
432     windows_total = len(samples)
433
434     # ---- cache setup / resume decision ----- #
435     if cache_dir is None:
436         cache_dir = default_cache_dir(out_csv)
437     os.makedirs(cache_dir, exist_ok=True)
438     det_jsonl, _ = _cache_paths(cache_dir)
439     cache_key = source_key if source_key is not None else osp.abspath(frames_dir)
440     manifest = None if fresh else read_manifest(cache_dir)
441     resume = manifest_compatible(
442         manifest or {}, frames_dir=cache_key, weights=weights, sport=sport,
443         frames_in=frames_in, frames_total=len(frames), device=device,
444     )
445     if resume:
446         windows_done, det_by_fid = load_and_clean_cache(cache_dir)
447         logger.info(f"resuming from cache: {windows_done}/{windows_total} windows
already detected")
448     else:
449         if manifest is not None:
450             logger.info("cache incompatible with this run -> starting fresh")
451             reset_cache(cache_dir)
452             windows_done, det_by_fid = 0, defaultdict(list)
453
454     base_manifest = {
455         "version": CACHE_VERSION,
456         "frames_dir": cache_key,
457         "weights": weights,
458         "sport": sport,
459         "frames_in": frames_in,
460         "frames_total": len(frames),
461         "device": device,
462         "windows_total": windows_total,
463         "windows_done": windows_done,
464     }
465     write_manifest(cache_dir, base_manifest)
466
467     # ---- detector pass over the un-cached windows ----- #
468     remaining = samples[windows_done:]
469     if remaining:
470         _, transform_test = build_img_transforms(cfg)
471         try:
472             _, seq_transform_test = build_seq_transforms(cfg)
473         except Exception:
474             seq_transform_test = None
475         ds = ImageDataset(cfg, remaining, input_wh, output_wh,
476             transform=transform_test, seq_transform=seq_transform_test,
is_train=False)
477         # Streaming forces single-process loading: the in-memory frame store + the
478         # cv2.imread shim live in THIS process and can't be pickled to DataLoader
workers.

```

```

479         loader_workers = 0 if streaming else num_workers
480         loader = DataLoader(ds, batch_size=batch_size, shuffle=False,
num_workers=loader_workers)
481         detector = build_detector(cfg)
482
483         from contextlib import ExitStack
484
485         t0 = time.time()
486         done = windows_done
487         win_idx = windows_done
488         log_every = max(1, log_every_batches)
489         flush_n = max(1, flush_every)
490         pending = []
491         logger.info(f"detecting on {len(remaining)} remaining windows "
492             f"(total {windows_total}, batch={batch_size},
workers={loader_workers}, "
493             f"source={'video-stream' if streaming else 'frames-dir'})...")
494         det_f = open(det_jsonl, "a")
495         try:
496             with ExitStack() as stack:
497                 stack.enter_context(torch.no_grad())
498                 # In streaming mode, route ImageDataset's per-frame `cv2.imread(path)`
to the
499                 # in-memory decoded frame for that synthetic path; every other line of
500                 # __getitem__ runs unchanged so the tensor matches the PNG round-trip
exactly.
501                 # The detector's first needed frame is `windows_done` (window i starts
at
502                 # frame i), so prime the store there for a resume.
503                 if streaming:
504                     stack.enter_context(
505                         _streaming_read_image_patch(frame_loader, len(frames),
506                                                         window=frames_in,
start_index=windows_done))
507                 for bi, batch in enumerate(loader):
508                     imgs, _hms, trans = batch[0], batch[1], batch[2]
509                     img_paths = [list(t) for t in batch[-1]] # [frame_in][batch] ->
path
510                     batch_results, _ = detector.run_tensor(imgs, trans)
511                     nb = imgs.shape[0]
512                     for ib in range(nb):
513                         frames_payload = []
514                         for ie in sorted(batch_results[ib].keys()):
515                             p = img_paths[ie][ib]
516                             fid = _frame_id(p)
517                             plain = [_det_plain(d) for d in batch_results[ib][ie]]
518                             frames_payload.append([fid, plain])
519                             det_by_fid[fid].extend(plain)
520                         pending.append({"w": win_idx, "f": frames_payload})
521                         win_idx += 1
522                     done += nb
523                 if len(pending) >= flush_n or done >= windows_total:
524                     _append_windows(det_f, pending)
525                     pending = []
526                     base_manifest["windows_done"] = done
527                     write_manifest(cache_dir, base_manifest)
528                 if (bi + 1) % log_every == 0 or done >= windows_total:
529                     el = time.time() - t0
530                     new_done = done - windows_done
531                     rate = new_done / el if el > 0 else 0.0
532                     eta = (windows_total - done) / rate if rate > 0 else 0.0
533                     logger.info(f"{done}/{windows_total} "
534                         f"({100 * done // max(1, windows_total)}%) |
{rate:.1f} win/s | "
535                         f"elapsed {el:.0f}s | ETA {eta:.0f}s")

```

```

536         if pending:
537             _append_windows(det_f, pending)
538             base_manifest["windows_done"] = done
539             write_manifest(cache_dir, base_manifest)
540         finally:
541             det_f.close()
542             logger.info(f"detection done in {time.time() - t0:.0f}s; running tracker...")
543     else:
544         logger.info("detector cache already complete -> tracker-only re-run")
545
546     # ---- tracker re-run over the full ordered cache (cheap, deterministic) ---- #
547     tracker = build_tracker(cfg)
548     tracker.refresh()
549     rows = []
550     for fid in sorted(det_by_fid.keys()):
551         r = tracker.update(_dets_to_tracker(det_by_fid[fid]))
552         rows.append((fid, 1 if r.get("visi") else 0, r.get("x", -1), r.get("y", -1)))
553
554     _atomic_write_trajectory(out_csv, rows)
555     visible = sum(1 for _, v, *_ in rows if v)
556     logger.info(f"wrote {len(rows)} rows -> {out_csv}")
557     logger.info(f"visible-shuttle frames: {visible}/{len(rows)}")
558     return out_csv
559
560
561 def extract_frames(video_path: str, frames_dir: str) -> str:
562     """Decode a video to PNG frames named by source frame index (00000000.png ...).
563
564     Idempotent: if the dir already holds the expected number of frames (matching the
565     container's reported frame count) we skip re-decoding ? re-extraction after a
566     reboot is wasteful. NOTE: cv2 PNG extraction is slow; for big clips prefer
567     extracting with ffmpeg on the host and passing --frames_dir.
568     """
569     import cv2
570     os.makedirs(frames_dir, exist_ok=True)
571     cap = cv2.VideoCapture(video_path)
572     if not cap.isOpened():
573         raise SystemExit(f"cannot open video: {video_path}")
574     expected = int(cap.get(cv2.CAP_PROP_FRAME_COUNT) or 0)
575     existing = len(glob.glob(osp.join(frames_dir, "*.png")))
576     if expected > 0 and existing >= expected:
577         cap.release()
578         logger.info(f"reusing {existing} already-extracted frames -> {frames_dir}")
579         return frames_dir
580     i = 0
581     while True:
582         ok, frame = cap.read()
583         if not ok:
584             break
585         cv2.imwrite(osp.join(frames_dir, f"{i:08d}.png"), frame)
586         i += 1
587     cap.release()
588     logger.info(f"extracted {i} frames -> {frames_dir}")
589     return frames_dir
590
591
592 def _probe_frame_count(video_path: str) -> int:
593     """Container-reported frame count via cv2 (0 if unknown). Used to size the
594     stream."""
595     import cv2
596     cap = cv2.VideoCapture(video_path)
597     if not cap.isOpened():
598         raise SystemExit(f"cannot open video: {video_path}")
599     n = int(cap.get(cv2.CAP_PROP_FRAME_COUNT) or 0)
600     cap.release()

```

```

600         return n
601
602
603     def make_video_frame_loader(video_path: str):
604         """Return ``(frame_loader, count_hint, release)`` for output-preserving streaming.
605
606         ``frame_loader(index) -> BGR uint8 ndarray`` decodes the video forward-only with one
607         long-lived ``cv2.VideoCapture``. It MUST be called with non-decreasing indices (the
608         detector's sliding-window order); it reads sequentially and only uses a frame-seek
609         to
610         skip forward when a *resume* starts past the current position. Each ``cap.read()``
611         returns exactly the BGR uint8 array that ``cv2.imwrite``/``cv2.imread`` of a PNG
612         would
613         yield (PNG is lossless), so the detector sees identical pixels to the disk path.
614
615         Returns the count hint from the container so the caller can build the frame list;
616         ``release`` closes the capture.
617         """
618         import cv2
619         cap = cv2.VideoCapture(video_path)
620         if not cap.isOpened():
621             raise SystemExit(f"cannot open video: {video_path}")
622         count_hint = int(cap.get(cv2.CAP_PROP_FRAME_COUNT) or 0)
623         state = {"pos": 0} # next index cap.read() will return
624
625         def frame_loader(index: int):
626             index = int(index)
627             if index < state["pos"]:
628                 raise RuntimeError(
629                     f"streaming decode is forward-only; got index {index} after
630 {state['pos']}")
631             if index > state["pos"]:
632                 # Forward skip only happens when resuming past cached windows. Seek once.
633                 cap.set(cv2.CAP_PROP_POS_FRAMES, index)
634                 state["pos"] = index
635                 ok, frame = cap.read()
636                 if not ok:
637                     raise RuntimeError(f"failed to decode frame {index} from {video_path}")
638                 state["pos"] += 1
639                 return frame
640
641         def release():
642             cap.release()
643
644         return frame_loader, count_hint, release
645
646     def run_video_streaming(video_path: str, weights: str, out_csv: str, sport: str =
647 "badminton",
648                             limit: int = 0, batch_size: int = 8,
649                             log_every_batches: int = 50, cache_dir: str | None = None,
650                             flush_every: int = 1000, fresh: bool = False,
651                             device: str = "cuda") -> str:
652         """Fast path: decode ``video_path`` on the fly and feed frames straight to the
653         detector, WITHOUT extracting every frame to a PNG first (the disk round-trip that
654         dominates a long clip). Output is bit-identical to the frames-on-disk path; see
655         ``run``'s docstring for why.
656         """
657         frame_loader, count_hint, release = make_video_frame_loader(video_path)
658         try:
659             if count_hint <= 0:
660                 raise SystemExit(
661                     f"could not determine frame count for {video_path}; cannot stream "
662                     f"(fall back to frame extraction with --frames_dir)")
663             frame_list = [synthetic_frame_path(i) for i in range(count_hint)]

```

```

661         source_key = "video:" + osp.abspath(video_path)
662         return run(frame_list, weights, out_csv, sport, limit,
663                   batch_size=batch_size, num_workers=0,
664                   log_every_batches=log_every_batches, cache_dir=cache_dir,
665                   flush_every=flush_every, fresh=fresh,
666                   frame_loader=frame_loader, source_key=source_key, device=device)
667     finally:
668         release()
669
670
671     def main():
672         ap = argparse.ArgumentParser()
673         ap.add_argument("--frames_dir", help="folder of pre-extracted frames")
674         ap.add_argument("--video", help="video file; frames are extracted internally (in
WSL)")
675         ap.add_argument("--frames_out_dir", help="where to extract frames when --video is
used")
676         ap.add_argument("--stream-video", action="store_true",
677                         help="FAST PATH: decode --video on the fly and feed frames straight
to "
678                             "the detector (no PNG extraction). Output-equivalent to the
disk path.")
679         ap.add_argument("--weights", required=True)
680         ap.add_argument("--out", required=True)
681         ap.add_argument("--sport", default="badminton")
682         ap.add_argument("--device", default="cuda", choices=["cuda", "mps", "cpu"],
683                         help="torch device for the detector (default cuda = the WSL parity
path)")
684         ap.add_argument("--limit", type=int, default=0, help="cap #frames (for quick
tests)")
685         ap.add_argument("--batch-size", type=int, default=8)
686         ap.add_argument("--num-workers", type=int, default=4)
687         ap.add_argument("--log-every-batches", type=int, default=50, help="progress log
cadence")
688         ap.add_argument("--cache-dir", default=None,
689                         help="resumable detector cache dir (default: <out>_wasbcache next to
--out)")
690         ap.add_argument("--flush-every", type=int, default=1000,
691                         help="fsync the detector cache every N windows (bounds reboot
loss)")
692         ap.add_argument("--fresh", action="store_true", help="ignore any existing cache and
start over")
693         args = ap.parse_args()
694
695         if args.stream_video:
696             if not args.video:
697                 raise SystemExit("--stream-video requires --video")
698             run_video_streaming(args.video, args.weights, args.out, args.sport, args.limit,
699                               batch_size=args.batch_size,
700                               log_every_batches=args.log_every_batches,
cache_dir=args.cache_dir,
701                               flush_every=args.flush_every, fresh=args.fresh,
device=args.device)
702             return
703
704         frames_dir = args.frames_dir
705         if args.video:
706             frames_dir = extract_frames(args.video, args.frames_out_dir or (args.video +
"_frames"))
707         if not frames_dir:
708             raise SystemExit("provide --frames_dir or --video")
709         run(frames_dir, args.weights, args.out, args.sport, args.limit,
710             batch_size=args.batch_size, num_workers=args.num_workers,
711             log_every_batches=args.log_every_batches, cache_dir=args.cache_dir,
712             flush_every=args.flush_every, fresh=args.fresh, device=args.device)

```

```

713
714
715     if __name__ == "__main__":
716         main()
717

```

8. user (2026-06-22T09:08:14.221Z)

```

1     """TrackNet shuttle-trajectory runner (WSL2 subprocess adapter).
2
3     This bridges the Windows-side indexer to a TensorFlow TrackNetV4 install living
4     inside WSL2 (ADR-001 in docs/DECISIONS.md). The flow is:
5
6         1. Translate the Windows video path to its WSL-visible form (and optionally
7           stage it into the WSL filesystem to avoid slow /mnt/c per-frame reads).
8         2. Invoke ``src/predict.py`` inside the TrackNetV4 conda env via ``wsl``.
9         3. Read back the per-frame ``Frame,Visibility,X,Y`` CSV it writes.
10        4. Convert that trajectory into high-motion "action windows" ? the same
11           candidate-rally shape HybridYoloSegmenter produces ? so the rest of the
12           pipeline (AI handoff, DB population) is unchanged.
13
14     Nothing here imports TensorFlow; all model code stays in WSL.
15     """
16
17     import os
18     import csv
19     import shlex
20     import logging
21     import subprocess
22     from dataclasses import dataclass
23     from typing import List, Dict, Any, Optional, Tuple
24
25     from backend.pipeline.detectors.base import DetectorRunner, WslCommandMixin,
wsl_tilde_quote
26
27     logger = logging.getLogger(__name__)
28
29
30     @dataclass
31     class TrackNetConfig:
32         """Resolved settings for a TrackNet WSL run (built from config.json ->
indexing.tracknet)."""
33         distro: str = "Ubuntu"
34         repo_dir: str = "~/models/TrackNetV4" # WSL path to the cloned repo
35         conda_sh: str = "~/miniconda3/etc/profile.d/conda.sh"
36         conda_env: str = "TrackNetV4"
37         weights_path: str = "" # WSL path to the .h5/.keras weights
38         (REQUIRED)
39         queue_length: int = 5
40         stage_in_wsl: bool = True # copy video into WSL fs first
41         (perf)
42         wsl_stage_dir: str = "~/clips" # where staged videos/outputs live
43         in WSL
44         timeout_sec: int = 1800 # 30 min; kills a hung call (0 = no
timeout)
45         keep_frames: bool = False # retain staged video after success
(debug only)
46         min_free_gb: float = 0.0 # warn if WSL free space below this
(0 = off)
47
48         @classmethod
49         def from_indexing_cfg(cls, idx_cfg: Dict[str, Any]) -> "TrackNetConfig":
50             tn = (idx_cfg or {}).get("tracknet", {}) or {}
51             return cls(
52                 distro=tn.get("wsl_distro", "Ubuntu"),

```

```

50         repo_dir=tn.get("repo_dir", "~/models/TrackNetV4"),
51         conda_sh=tn.get("conda_sh", "~/miniconda3/etc/profile.d/conda.sh"),
52         conda_env=tn.get("conda_env", "TrackNetV4"),
53         weights_path=tn.get("weights_path", ""),
54         queue_length=int(tn.get("queue_length", 5)),
55         stage_in_wsl=bool(tn.get("stage_in_wsl", True)),
56         wsl_stage_dir=tn.get("wsl_stage_dir", "~/clips"),
57         timeout_sec=int(tn.get("timeout_sec", 1800)),
58         keep_frames=bool(tn.get("keep_frames", False)),
59         min_free_gb=float(tn.get("min_free_gb", 0.0)),
60     )
61
62
63 def to_wsl_mnt_path(win_path: str) -> str:
64     """Translate a Windows path (C:\\Users\\x) to its WSL /mnt form (/mnt/c/Users/x).
65
66     Already-POSIX paths (starting with / or ~) are returned unchanged so the
67     same helper is safe to call on either kind of path.
68     """
69     p = str(win_path)
70     if p.startswith("/") or p.startswith("~"):
71         return p
72     p = p.replace("\\", "/")
73     if len(p) >= 2 and p[1] == ":":
74         drive = p[0].lower()
75         return f"/mnt/{drive}{p[2:]}"
76     return p
77
78
79 @dataclass
80 class TrajectoryPoint:
81     frame: int
82     visible: bool
83     x: float
84     y: float
85
86
87 class TrackNetRunner(WslCommandMixin, DetectorRunner):
88     """Runs TrackNet predict.py in WSL and turns the output CSV into action windows.
89
90     Implements the common DetectorRunner contract so a future Linux-native or
91     alternative trajectory runner can be substituted without touching the hybrids.
92     """
93
94     def __init__(self, cfg: TrackNetConfig):
95         self.cfg = cfg
96
97     def healthcheck(self) -> Tuple[bool, str]:
98         """Verify the WSL env is usable: conda env exists, TF imports, GPU visible.
99
100         Returns (ok, message). Cheap to call before a long run.
101         """
102         cmd = (
103             f"conda activate {self.cfg.conda_env} && "
104             f"python -c \"import tensorflow as tf; "
105             f"print('TF', tf.__version__, 'GPUs', "
106             f"len(tf.config.list_physical_devices('GPU')))\n"
107         )
108         try:
109             res = self._wsl_bash(cmd)
110         except FileNotFoundError:
111             return False, "wsl` not found ? is this running on Windows with WSL2 installed?"
112         except subprocess.TimeoutExpired:
113             return False, "WSL healthcheck timed out."

```



```

113         out = (res.stdout or "").strip()
114         if res.returncode != 0:
115             return False, f"WSL env check failed: {(res.stderr or out).strip()}"
116         return True, out
117
118 # ----- inference
119
120 def run_predict(self, video_win_path: str, output_win_dir: str,
121                 video_id: Optional[str] = None) -> Optional[str]:
122     """Run predict.py on a video. Returns the Windows path to the produced CSV, or
123     None.
124
125     ``output_win_dir`` is a Windows directory (under the repo's output/) that
126     WSL writes into via its /mnt view, so the Windows process can read the CSV
127     back with no copy. ``video_id`` (file MD5) namespaces the staged video ? and
128     therefore predict.py's ``<stem>_predict.csv`` output ? so two videos that share
129     a filename cannot collide on the output CSV.
130     """
131     if not self.cfg.weights_path:
132         logger.error(
133             "TrackNet weights_path is not set. Set indexing.tracknet.weights_path "
134             "in config.json to the WSL path of your trained TrackNetV4 weights."
135         )
136         return None
137
138     # Resolve relative dirs BEFORE the /mnt translation ? to_wsl_mnt_path passes
139     # relative paths through unchanged, so WSL would resolve them against the
140     # predict.py cwd instead of the repo (same bug class as wasb_runner).
141     output_win_dir = os.path.abspath(output_win_dir)
142     os.makedirs(output_win_dir, exist_ok=True)
143     # Normalize separators so basename/splittest work when tests (or collector)
144     # pass Windows-style paths on a Linux host.
145     video_win_path = str(video_win_path).replace("\\", "/")
146     base = os.path.basename(video_win_path)
147     stem, ext = os.path.splitext(base)
148
149     # predict.py only accepts .avi/.mp4 and names the CSV "<stem>_predict.csv".
150     if ext.lower() not in (".mp4", ".avi"):
151         logger.error("TrackNet predict.py only supports .mp4/.avi (got %s)", ext)
152         return None
153
154     wsl_out = to_wsl_mnt_path(output_win_dir)
155     # Namespace by video_id so two same-filename videos don't collide on the CSV.
156     # predict.py derives its output name from the (staged) video stem, so staging
157     # under the namespaced name propagates into "<key>_predict.csv".
158     key = f"{stem}_{video_id[:12]}" if video_id else stem
159     expected_csv_win = os.path.join(output_win_dir, f"{key}_predict.csv")
160
161     # Resolve the video path WSL will read.
162     if self.cfg.stage_in_wsl:
163         staged = self._stage_video(video_win_path, f"{key}{ext}")
164         if staged is None:
165             return None
166         wsl_video = staged
167     else:
168         # No staging: predict.py reads /mnt directly and writes
169         # "<stem>_predict.csv".
170         # We cannot rename its output, so fall back to the un-namespaced name.
171         wsl_video = to_wsl_mnt_path(video_win_path)
172         expected_csv_win = os.path.join(output_win_dir, f"{stem}_predict.csv")
173
174     if self.cfg.min_free_gb > 0:
175         free = self._wsl_free_gb(self.cfg.wsl_stage_dir)
176         if free is not None and free < self.cfg.min_free_gb:
177             logger.warning("Low WSL disk: %.1f GB free at %s (< min_free_gb=%.1f); "

```

```

176                 "consider running tools/cleanup_caches.py.",
177                 free, self.cfg.wsl_stage_dir, self.cfg.min_free_gb)
178
179         cmd = (
180             f"conda activate {self.cfg.conda_env} && "
181             f"cd {self.cfg.repo_dir}/src && "
182             f"python predict.py "
183             f"--video_path {wsl_tilde_quote(wsl_video)} "
184             f"--model_weights {wsl_tilde_quote(self.cfg.weights_path)} "
185             f"--output_dir {wsl_tilde_quote(wsl_out)} "
186             f"--queue_length {self.cfg.queue_length}"
187         )
188         logger.info("Running TrackNet inference on %s ...", base)
189         try:
190             res = self._wsl_bash(cmd)
191         except subprocess.TimeoutExpired:
192             logger.error("TrackNet inference timed out after %ss.",
self.cfg.timeout_sec)
193             return None
194
195         if res.returncode != 0:
196             logger.error("TrackNet predict.py failed:\n%s", (res.stderr or
res.stdout)[-2000:])
197             return None
198
199         if not os.path.exists(expected_csv_win):
200             logger.error("TrackNet finished but expected CSV not found at %s",
expected_csv_win)
201             return None
202         logger.info("TrackNet trajectory CSV: %s", expected_csv_win)
203
204         # Success ? the CSV is the durable output; drop the staged video copy (a pure,
205         # regenerable intermediate). On earlier failure we return above without
cleaning.
206         if self.cfg.stage_in_wsl and not self.cfg.keep_frames:
207             logger.info("Cache hygiene: removing staged video for %s "
208                         "(set indexing.tracknet.keep_frames=true to retain).", key)
209             self._wsl_rm_rf([wsl_video])
210         return expected_csv_win
211
212     def _stage_video(self, video_win_path: str, base: str) -> Optional[str]:
213         """Copy the input video into the WSL filesystem (avoids slow /mnt/c reads).
214
215         Returns the WSL path to the staged copy, or None on failure.
216         """
217         src_mnt = to_wsl_mnt_path(video_win_path)
218         reason = self.wsl_source_unreadable_reason(src_mnt)
219         if reason:
220             logger.error("Cannot stage video into WSL: %s", reason)
221             return None
222         dest = f"{self.cfg.wsl_stage_dir}/{base}"
223         cmd = f"mkdir -p {self.cfg.wsl_stage_dir} && cp {shlex.quote(src_mnt)}
{wsl_tilde_quote(dest)} && echo OK"
224         try:
225             res = self._wsl_bash(cmd)
226         except subprocess.TimeoutExpired:
227             logger.error("Staging video into WSL timed out.")
228             return None
229         if res.returncode != 0 or "OK" not in (res.stdout or ""):
230             logger.error("Failed to stage video into WSL: %s", (res.stderr or
res.stdout).strip())
231             return None
232         return dest
233
234     # ----- CSV -> windows

```

```

235
236 @staticmethod
237 def parse_trajectory_csv(csv_win_path: str) -> List[TrajectoryPoint]:
238     """Parse a TrackNet predict CSV (columns: Frame,Visibility,X,Y)."""
239     points: List[TrajectoryPoint] = []
240     with open(csv_win_path, newline="") as f:
241         reader = csv.DictReader(f)
242         for row in reader:
243             try:
244                 points.append(TrajectoryPoint(
245                     frame=int(row["Frame"]),
246                     visible=int(row["Visibility"]) == 1,
247                     x=float(row["X"]),
248                     y=float(row["Y"]),
249                 ))
250             except (KeyError, ValueError):
251                 continue
252     points.sort(key=lambda p: p.frame)
253     return points
254
255 @staticmethod
256 def _active_frames(points: List[TrajectoryPoint], fps: float, frame_width: float,
257                    velocity_thresh: float) -> List[Tuple[int, float]]:
258     """Per-frame ``(frame_idx, velocity)`` for frames where the shuttle is in
flight.
259
260     Computes each visible point's normalised velocity from the last *visible* point;
261     an absent shuttle breaks the trajectory. A frame qualifies when its velocity
exceeds
262     ``velocity_thresh``. ``fps``/``frame_width`` are expected to be already
defaulted by
263     the caller so identical floats feed the velocity math."""
264     active = [] # (frame_idx, velocity)
265     prev: Optional[TrajectoryPoint] = None
266     for p in points:
267         if not p.visible:
268             prev = None # break the trajectory; absent shuttle = not in flight
269             continue
270         if prev is not None:
271             dframes = p.frame - prev.frame
272             if dframes > 0:
273                 dist = ((p.x - prev.x) ** 2 + (p.y - prev.y) ** 2) ** 0.5
274                 velocity = (dist / frame_width) * (fps / dframes)
275                 if velocity > velocity_thresh:
276                     active.append((p.frame, velocity))
277         prev = p
278     return active
279
280 @staticmethod
281 def trajectory_to_action_windows(
282     points: List[TrajectoryPoint],
283     fps: float,
284     frame_width: float,
285     chunk_start: float = 0.0,
286     velocity_thresh: float = 0.02,
287     merge_gap: float = 2.0,
288     min_window_duration: float = 2.0,
289     chunk_index: Optional[int] = None,
290     max_window_duration: float = 0.0,
291     min_split_gap: float = 0.5,
292     window_hard_cap: bool = False,
293     window_inactivity_end: float = 0.0,
294     inactivity_min_density: float = 0.34,
295     inactivity_rally_thresh: float = 0.08,
296     window_blob_fallback: bool = False,

```

```

297         window_blob_factor: float = 4.0,
298     ) -> List[Dict[str, Any]]:
299         """Convert a shuttle trajectory into candidate rally windows.
300
301         A frame is "active" when the shuttle is visible AND its inter-frame
302         displacement (normalised by frame width, per second) exceeds
303         ``velocity_thresh`` ? i.e. the shuttle is in flight. Active frames within
304         ``merge_gap`` seconds of each other are grouped into one window; short
305         windows are dropped. Mirrors HybridYoloSegmenter's windowing so the dict
306         shape feeding the AI-handoff phase is identical.
307
308         ``max_window_duration`` (0 = OFF, the default ? behaviour unchanged): a guard
309         against the *over-merge* failure where "shuttle moving" is conflated with "rally in
310         progress" and a single window swallows several rallies + the dead time between them. When
311         set, any window longer than this is recursively SPLIT at its largest internal inactivity
312         gap (the longest stretch with no in-flight shuttle ? the most likely rally boundary),
313         provided that gap is at least ``min_split_gap`` seconds. This is the bounded-windowing
314         fix from ``docs/RALLY_QUALITY_RESEARCH.md`` §6 (W1); it never merges, only cuts, so
315         recall cannot drop, and it is config-gated default-OFF until measured on the golden set.
316
317         ``window_hard_cap`` (default OFF) makes ``max_window_duration`` a TRUE cap: when
318         an over-long window has NO internal inactivity gap to split on (a continuously-
319         active trajectory ? the shuttle never stops moving, e.g. the real-footage
320         ``mahadevpura-1`` blob where the gap-splitter alone left a single 174 s window), it is recursively
321         hard-chopped at its temporal midpoint until every piece is ? the cap. Still only
322         cuts (never merges), so recall cannot drop; gated default-OFF until measured.
323
324         ``window_blob_fallback`` (default OFF ? the R5 last-resort blob splitter):
325         unlike ``window_hard_cap`` (which midpoint-chops BEFORE the R4 trim, so it fires on
326         every gap-less over-cap window and over-segments normal clips), this runs AFTER the R4
327         inactivity trim and force-chops ONLY a genuine *blob* ? a group still longer than
328         ``window_blob_factor`` × ``max_window_duration`` once the gap-split and the principled rally-end trim
329         have both had their chance (e.g. mahadevpura-1's ~65 s residual ? 11× a 6 s cap; a 7 s rally
330         is left alone). The blob is midpoint-chopped down to ? the cap, those forced cuts are
331         logged, and each resulting window is flagged ``forced_cut=True`` ? surfacing the clip as one
332         that needs rally-STATE cues (net-crossings / two-sided motion), not a bigger cap. Only cuts
333         (recall can't drop). Requires ``max_window_duration`` > 0; gated default-OFF until
334         measured.
335
336         """
337         if fps <= 0:
338             fps = 30.0
339         if frame_width <= 0:
340             frame_width = 1280.0
341
342         active = TrackNetRunner._active_frames(points, fps, frame_width,

```

```

velocity_thresh)
341
342         if not active:
343             return []
344
345         max_gap_frames = int(merge_gap * fps)
346
347         def _finalize(group: List[Tuple[int, float]]) -> Dict[str, Any]:
348             vels = [v for _, v in group]
349             return {
350                 "start": group[0][0] / fps + chunk_start,
351                 "end": group[-1][0] / fps + chunk_start,
352                 "active_frame_count": len(group),
353                 "peak_velocity": max(vels),
354                 "mean_velocity": sum(vels) / len(vels),
355                 "track_id_count": 1, # single shuttle; kept for window-shape parity
356                 "chunk_index": chunk_index,
357             }
358
359         groups: List[List[Tuple[int, float]]] = []
360         group = [active[0]]
361         for af in active[1:]:
362             if af[0] - group[-1][0] <= max_gap_frames:
363                 group.append(af)
364             else:
365                 groups.append(group)
366                 group = [af]
367         groups.append(group)
368
369         if max_window_duration > 0:
370             groups = TrackNetRunner._split_overlong_groups(
371                 groups, int(max_window_duration * fps), int(min_split_gap * fps),
372                 hard_cap=window_hard_cap)
373
374         # R4 ? rally-END inactivity trim (default OFF). Applied AFTER the cap split so
each rally
375         # piece has its own post-rally drift tail removed: the velocity windowing keeps
a window
376         # "active" while the shuttle drifts/gets-picked-up above threshold, over-
extending the END
377         # (the measured gap vs golden). Trim back to the last frame whose trailing
window is still
378         # dense with motion (rally on); a sustained low-density run = rally over. Only
cuts.
379         if window_inactivity_end > 0:
380             tw = int(window_inactivity_end * fps)
381             groups = [TrackNetRunner._trim_inactive_tail(g, tw, inactivity_min_density,
382                 inactivity_rally_thresh) for g
in groups]
383
384         # R5 ? blob-fallback (default OFF). LAST resort: after the gap-split AND the R4
trim, any
385         # group still longer than the cap is a gap-less, rally-end-signal-less blob;
force-chop it to
386         # ? the cap at the midpoint and flag every piece, so the degenerate single-
window outcome is
387         # avoided and the clip is surfaced (logged) as needing rally-STATE cues, not a
bigger cap.
388         forced_flags = [False] * len(groups)
389         if window_blob_fallback and max_window_duration > 0:
390             groups, forced_flags = TrackNetRunner._blob_fallback_chop(
391                 groups, int(max_window_duration * fps), window_blob_factor)
392
393         windows = []
394         for g, forced in zip(groups, forced_flags):

```

```

395         w = _finalize(g)
396         if forced:
397             w["forced_cut"] = True
398         windows.append(w)
399     return [w for w in windows if (w["end"] - w["start"]) >= min_window_duration]
400
401     @staticmethod
402     def _trim_inactive_tail(group: List[Tuple[int, float]], trail_frames: int,
403                             min_density: float, rally_thresh: float) -> List[Tuple[int,
404 float]]:
405         """R4: trim the post-rally drift tail. After a rally the shuttle keeps moving
406         *slowly*
407         (picked up / walked / knock-up) above ``velocity_thresh``, so the velocity
408         windowing
409         over-extends the END. A frame is "fast" (real rally motion) iff its velocity >
410         ``rally_thresh``
411         (higher than the active threshold). The rally is "on" at frame ``i`` while its
412         trailing
413         ``trail_frames`` window holds >= ``min_density`` fraction of fast frames; trim
414         back to the
415         LAST such frame ? everything after is sustained slow drift (rally over). Only
416         cuts (recall
417         can't rise); a window whose tail never goes slow is untouched, and if no fast-
418         dense point
419         exists at all the group is kept whole (fail-safe, no over-trim). O(n) two-
420         pointer; the START
421         is left alone (already ~Gemini-accurate per the n=2 boundary-error finding)."""
422         if trail_frames <= 0 or len(group) < 2:
423             return group
424         frames = [f for f, _ in group]
425         fast = [1 if v > rally_thresh else 0 for _, v in group]
426         lo = 0
427         fast_sum = 0
428         last_dense = -1
429         for i in range(len(frames)):
430             fast_sum += fast[i]
431             while frames[lo] <= frames[i] - trail_frames:
432                 fast_sum -= fast[lo]
433                 lo += 1
434             win_count = i - lo + 1
435             if win_count > 0 and fast_sum / win_count >= min_density:
436                 last_dense = i
437         return group[: last_dense + 1] if last_dense >= 0 else group
438
439     @staticmethod
440     def _midpoint_split_index(group: List[Tuple[int, float]]) -> int:
441         """Index of the active frame nearest the group's temporal midpoint (used to chop
442         a
443         gap-less group into two). Shared by the W1 hard-cap fallback and the R5 blob-
444         fallback."""
445         mid = (group[0][0] + group[-1][0]) / 2.0
446         split_i = min(range(1, len(group)), key=lambda i: abs(group[i][0] - mid))
447         return split_i
448
449     @staticmethod
450     def _split_overlong_groups(groups: List[List[Tuple[int, float]]], max_win_frames:
451 int,
452 min_split_frames: int,
453 hard_cap: bool = False) -> List[List[Tuple[int, float]]]:
454         """Recursively split any active-frame group longer than ``max_win_frames`` at
455         its largest
456         internal gap (? ``min_split_frames``) ? an inactivity-aware cut at the most
457         likely rally
458         boundary. Splitting only subdivides existing groups (never merges/extends), so
459         it cannot

```

```

445         invent or drop coverage; a group with no gap ? the floor is left intact (a
genuinely long
446         rally) UNLESS ``hard_cap`` is set, in which case such a group is hard-chopped at
its
447         temporal midpoint until every piece is ? ``max_win_frames`` (makes the cap a
true upper
448         bound on continuously-active trajectories). Iterative worklist to bound stack
depth. """
449         if max_win_frames <= 0:
450             return groups
451         out: List[List[Tuple[int, float]]] = []
452         work = list(groups)
453         while work:
454             g = work.pop()
455             if len(g) < 2 or (g[-1][0] - g[0][0]) <= max_win_frames:
456                 out.append(g)
457                 continue
458             best_i, best_gap = -1, -1
459             for i in range(1, len(g)):
460                 gap = g[i][0] - g[i - 1][0]
461                 if gap > best_gap:
462                     best_gap, best_i = gap, i
463             if best_i < 0 or best_gap < min_split_frames:
464                 # Too long but no real dead-time gap. Default: keep as one (a genuine
long rally).
465                 # hard_cap: chop at the temporal midpoint so the cap is actually
enforced.
466                 if hard_cap:
467                     split_i = TrackNetRunner._midpoint_split_index(g)
468                     work.append(g[:split_i])
469                     work.append(g[split_i:])
470                 else:
471                     out.append(g)
472                     continue
473                 work.append(g[:best_i])
474                 work.append(g[best_i:])
475             out.sort(key=lambda grp: grp[0][0])
476             return out
477
478     @staticmethod
479     def _blob_fallback_chop(groups: List[List[Tuple[int, float]]], max_win_frames: int,
480                             blob_factor: float = 4.0
481                             ) -> Tuple[List[List[Tuple[int, float]]], List[bool]]:
482         """R5 blob splitter ? the gap-less, R4-survived residual. After the W1 gap-split
AND the R4
483         inactivity trim have each had their chance, a group still longer than
``blob_factor`` x
484         ``max_win_frames`` is a genuine continuously-active blob (no internal gap, no
rally-end
485         signal the principled rules could find ? the real-footage ``mahadevpura-1`` ~65
s residual ?
486         11x a 6 s cap). The factor is what keeps this SURGICAL: a normal long rally
(~1?2x the cap)
487         is left ALONE; only the degenerate blob qualifies. As a LAST RESORT the blob is
midpoint-chopped to ? the cap and every resulting piece is FLAGGED as a forced
cut, so the
488         degenerate single-window outcome is avoided and the clip is surfaced (logged) as
needing
489         rally-STATE cues ? net-crossings / two-sided motion ? not a bigger cap. Only
cuts (recall
490         can't drop). Returns ``(groups, forced_flags)`` aligned 1:1; ``forced_flags[i]``
is True iff
491         that piece came from a forced chop. Iterative worklist to bound stack depth. """
492         if max_win_frames <= 0:
493             return groups, [False] * len(groups)
494

```

```

495         blob_frames = max_win_frames * max(1.0, blob_factor)
496         out: List[List[Tuple[int, float]]] = []
497         flags: List[bool] = []
498         forced = 0
499         for g in groups:
500             if len(g) < 2 or (g[-1][0] - g[0][0]) <= blob_frames:
501                 out.append(g)
502                 flags.append(False)
503                 continue
504             pieces: List[List[Tuple[int, float]]] = []
505             work = [g]
506             while work:
507                 h = work.pop()
508                 if len(h) < 2 or (h[-1][0] - h[0][0]) <= max_win_frames:
509                     pieces.append(h)
510                     continue
511                 split_i = TrackNetRunner._midpoint_split_index(h)
512                 work.append(h[:split_i])
513                 work.append(h[split_i:])
514             pieces.sort(key=lambda grp: grp[0][0])
515             out.extend(pieces)
516             flags.extend([True] * len(pieces))
517             forced += len(pieces) - 1
518         if forced:
519             logger.warning(
520                 "R5 blob-fallback: forced %d midpoint cut(s) on continuously-active
group(s) with "
521                 "no rally-end signal ? this clip needs rally-STATE cues (net-crossings /
two-sided "
522                 "motion), not a bigger cap", forced)
523         return out, flags
524

```

9. user (2026-06-22T09:08:14.540Z)

```

1         """Single-sourced candidate-windowing presets ? the eval?serve foundation fix.
2
3         **The bug this closes.** The offline eval substrate (`distill_local.build_candidates`
4         ?
5         ``fusion_golden`` ? the golden feature JSONs the ablation/compare path re-scores) was
built
6         on a *recall-oriented PROPOSAL* windowing (`vt=0.005, merge_gap=1.0, min_window=0.5`):
7         deliberately permissive so a learned scorer/gate can filter the junk. But the SERVED
pipeline
8         (`TrajectoryHybridSegmenter.process_video`) windows COARSELY (`vt=0.02,
merge_gap=2.0,
9         min_window=2.0` ? the `config.indexing` defaults). A cue measured on the proposal set
is
10        measured on candidates **production never serves**, so a "win" need not transfer. This
is
11        exactly how Gate-A scored **0.074** on the proposal set yet **0.008** on the served
set
12        (`scratch/GATE_AB_NUMBERS_AND_IMPLICATIONS.md` vs `output/gateA_served_verify`).
13
14        **The fix.** One module owns the two named windowings as :class:`WindowingPreset`, and
the
15        SERVED preset is **single-sourced from the same Pydantic config defaults production
reads**
16        (:class:`backend.config.models.IndexingConfig`) ? eval and serve can no longer drift.
Helpers
17        build candidates under either preset, and :func:`served_windowing_from_config` lifts the
18        *actual* served windowing out of a live config (overrides included), so an A/B can
target the
19        operating point a given deployment truly serves.

```



```

20     The companion rule ? *a cue is promotable only if it does NOT regress at the SERVED
21     windowing* ? lives in :mod:`backend.eval.promotion` (which composes ``eval_stats`` +
22     ``per_user_gate``). This module is the what windowing; that one is the promotion
gate**.
23
24     Pure, offline, stdlib + config only. Nothing here changes existing eval behaviour until
a
25     caller opts in (``distill_local`` re-points its module-level presets here, byte-
identically).
26     """
27     from __future__ import annotations
28
29     from dataclasses import dataclass
30     from typing import Any, Dict, List, Mapping, Tuple
31
32     from backend.config.models import IndexingConfig
33
34     Interval = Tuple[float, float]
35
36     __all__ = [
37         "WindowingPreset",
38         "SERVED",
39         "PROPOSAL",
40         "PRESETS",
41         "served_windowing_from_config",
42         "build_windows",
43         "windows_to_intervals",
44     ]
45
46
47     @dataclass(frozen=True)
48     class WindowingPreset:
49         """A named (velocity_thresh, merge_gap, min_window_duration) windowing operating
point.
50
51         ``velocity_thresh`` ? normalised per-second shuttle velocity above which a frame
counts as
52         "in flight"; ``merge_gap`` ? seconds within which active frames coalesce into one
window;
53         ``min_window_duration`` ? windows shorter than this are dropped. These are exactly
the three
54         knobs ``TrackNetRunner.trajectory_to_action_windows`` takes, so a preset is the
windowing.
55         """
56
57         name: str
58         velocity_thresh: float
59         merge_gap: float
60         min_window_duration: float
61         #: 0 = OFF (default). When > 0, windows longer than this are split at their largest
internal
62         #: inactivity gap ? the bounded-windowing over-merge fix (W1,
RALLY_QUALITY_RESEARCH.md §6).
63         max_window_duration: float = 0.0
64         #: When True, ``max_window_duration`` is a HARD cap ? an over-long window with no
internal
65         #: inactivity gap is chopped at its midpoint until ? the cap (continuously-active
fix). OFF by default.
66         hard_cap: bool = False
67         #: R4 rally-END inactivity trim (s, 0 = OFF): trim a window's post-rally slow-drift
tail back to
68         #: the last frame whose trailing window stays dense with FAST motion. The measured
rally-end fix.
69         inactivity_end: float = 0.0
70         #: velocity above which motion counts as "rally" (vs slow drift) for the R4 trim;

```

```

min fast-fraction.
71     inactivity_rally_thresh: float = 0.08
72     inactivity_min_density: float = 0.34
73     #: R5 blob-fallback (default OFF). After the cap split + R4 trim, midpoint-chop any
group still
74     #: longer than ``blob_factor`` × ``max_window_duration`` (the gap-less, R4-survived
blob ?
75     #: mahadevpura-1's ~11× residual) and flag it. Runs AFTER R4 (vs ``hard_cap``
before) and the
76     #: factor keeps it surgical (a normal long rally is left alone).
77     blob_fallback: bool = False
78     blob_factor: float = 4.0
79
80     def as_tuple(self) -> Tuple[float, float, float]:
81         """``(velocity_thresh, merge_gap, min_window_duration)`` ? the legacy tuple
shape the
82         ``trajectory_to_action_windows`` / ``distill_local`` call sites already speak.
(The
83         bounded-windowing knob ``max_window_duration`` is passed separately by
``build_windows``.)"""
84         return (self.velocity_thresh, self.merge_gap, self.min_window_duration)
85
86     def to_dict(self) -> Dict[str, Any]:
87         return {
88             "name": self.name,
89             "velocity_thresh": self.velocity_thresh,
90             "merge_gap": self.merge_gap,
91             "min_window_duration": self.min_window_duration,
92             "max_window_duration": self.max_window_duration,
93             "hard_cap": self.hard_cap,
94             "inactivity_end": self.inactivity_end,
95             "inactivity_rally_thresh": self.inactivity_rally_thresh,
96             "inactivity_min_density": self.inactivity_min_density,
97             "blob_fallback": self.blob_fallback,
98             "blob_factor": self.blob_factor,
99         }
100
101
102     def _served_preset_from_defaults() -> WindowingPreset:
103         """The SERVED windowing, lifted from the SAME Pydantic config defaults production
reads.
104
105         ``TrajectoryHybridSegmenter.process_video`` reads exactly these three:
106         - velocity: ``idx_cfg[<family>].velocity_threshold`` (default 0.02 ?
wasb/fusion/tracknet)
107         - merge_gap: ``idx_cfg.dead_time_merge_gap`` (default 2.0)
108         - min window: ``idx_cfg.min_action_window_duration`` (default 2.0)
109
110         Sourcing them from :class:`IndexingConfig`'s defaults (not a hand-typed tuple) is
the whole
111         point: change a served default in ``config/models.py`` and the eval SERVED preset
follows,
112         so the two can never silently diverge again.
113         """
114         idx = IndexingConfig() # construct with defaults ? the served operating point
115         return WindowingPreset(
116             name="served",
117             velocity_thresh=float(idx.wasb.velocity_threshold), # == fusion/tracknet
default 0.02
118             merge_gap=float(idx.dead_time_merge_gap),
119             min_window_duration=float(idx.min_action_window_duration),
120             max_window_duration=float(idx.max_window_duration), # W1 bounded-windowing
(default 0=OFF)
121             hard_cap=bool(idx.window_hard_cap), # W1 hard-cap fallback
(default OFF)

```

```

122         inactivity_end=float(idx.window_inactivity_end),      # R4 rally-end trim
(default 0=OFF)
123         inactivity_rally_thresh=float(idx.inactivity_rally_thresh),
124         inactivity_min_density=float(idx.inactivity_min_density),
125         blob_fallback=bool(idx.window_blob_fallback),          # R5 blob-fallback
(default OFF)
126         blob_factor=float(idx.window_blob_factor),
127     )
128
129
130     #: The COARSE windowing production actually serves (config defaults; ~0.02/2.0/2.0). A
cue is
131     #: only promotable if it holds up HERE ? the operating point real users get.
132     SERVED: WindowingPreset = _served_preset_from_defaults()
133
134     #: The recall-oriented PROPOSAL windowing the offline substrate is built on
(deliberately
135     #: permissive: propose many, let the classifier/gate filter). NOT what production serves
? a
136     #: win here must be re-confirmed on SERVED before promotion. Mirrors the historical
137     #: ``distill_local.PROPOSAL`` tuple (0.005/1.0/0.5) so the existing golden JSONs stay
valid.
138     PROPOSAL: WindowingPreset = WindowingPreset(
139         name="proposal", velocity_thresh=0.005, merge_gap=1.0, min_window_duration=0.5
140     )
141
142     #: Name ? preset, so a CLI / config string ("served" | "proposal") selects one.
143     PRESETS: Dict[str, WindowingPreset] = {SERVED.name: SERVED, PROPOSAL.name: PROPOSAL}
144
145
146     def served_windowing_from_config(config: Any, family: str = "wasb") -> WindowingPreset:
147         """The SERVED windowing a *specific* live config serves (overrides honoured).
148
149         Reproduces ``TrajectoryHybridSegmenter.process_video``'s knob reads exactly, against
an
150         arbitrary config (the typed :class:`AppConfig` or a plain dict) and segmenter
``family``
151         (``"wasb"`` / ``"fusion"`` / ``"tracknet"``). Use this when a deployment overrode a
served
152         default in ``config.json`` ? the module-level :data:`SERVED` is the *default*
operating point;
153         this is *this config's* operating point. Falls back to the served defaults for any
absent key.
154         """
155         idx = _as_mapping(config, "indexing")
156         family_cfg = _as_mapping(idx, family)
157         return WindowingPreset(
158             name=f"served:{family}",
159             velocity_thresh=float(family_cfg.get("velocity_threshold",
SERVED.velocity_thresh)),
160             merge_gap=float(idx.get("dead_time_merge_gap", SERVED.merge_gap)),
161             min_window_duration=float(idx.get("min_action_window_duration",
SERVED.min_window_duration)),
162             max_window_duration=float(idx.get("max_window_duration",
SERVED.max_window_duration)),
163             hard_cap=bool(idx.get("window_hard_cap", SERVED.hard_cap)),
164             inactivity_end=float(idx.get("window_inactivity_end", SERVED.inactivity_end)),
165             inactivity_rally_thresh=float(idx.get("inactivity_rally_thresh",
SERVED.inactivity_rally_thresh)),
166             inactivity_min_density=float(idx.get("inactivity_min_density",
SERVED.inactivity_min_density)),
167             blob_fallback=bool(idx.get("window_blob_fallback", SERVED.blob_fallback)),
168             blob_factor=float(idx.get("window_blob_factor", SERVED.blob_factor)),
169         )
170

```

```

171
172     def _as_mapping(obj: Any, key: str) -> Mapping[str, Any]:
173         """`obj[key]` as a plain mapping, whether `obj` is a dict or a typed config
section
174         (`AppConfig.get` returns nested sections as dicts). Empty mapping if absent/non-
mapping."""
175         if obj is None:
176             return {}
177         if isinstance(obj, Mapping):
178             sub = obj.get(key, {})
179         elif hasattr(obj, "get"):
180             sub = obj.get(key, {})
181         else:
182             sub = getattr(obj, key, {})
183         return sub if isinstance(sub, Mapping) else {}
184
185
186     def build_windows(points: List[Any], fps: float, frame_width: float,
187                       preset: WindowingPreset) -> List[Dict[str, Any]]:
188         """Trajectory `points` ? candidate windows under `preset` (the one windowing
call site).
189
190         A thin wrapper over `TrackNetRunner.trajectory_to_action_windows` that takes a
named preset
191         instead of three loose floats, so eval candidate-building and the served path
reference the
192         SAME knobs by name. Imported lazily to keep this module import-light (no detector
deps)."""
193         from backend.pipeline.detectors.tracknet_runner import TrackNetRunner
194
195         vt, mg, mwd = preset.as_tuple()
196         return TrackNetRunner.trajectory_to_action_windows(
197             points, fps=fps, frame_width=frame_width,
198             velocity_thresh=vt, merge_gap=mg, min_window_duration=mwd,
199             max_window_duration=preset.max_window_duration,
200             window_hard_cap=preset.hard_cap,
201             window_inactivity_end=preset.inactivity_end,
202             inactivity_rally_thresh=preset.inactivity_rally_thresh,
203             inactivity_min_density=preset.inactivity_min_density,
204             window_blob_fallback=preset.blob_fallback,
205             window_blob_factor=preset.blob_factor)
206
207
208     def windows_to_intervals(windows: List[Dict[str, Any]]) -> List[Interval]:
209         """`[{start, end, ...}]` ? `[(start, end)]` ? the interval shape the scorer/gate
speak."""
210         return [(w["start"], w["end"]) for w in windows]
211

```

10. assistant (2026-06-22T09:08:16.927Z)

Now let me examine the segmenter and stitching logic:

11. user (2026-06-22T09:08:18.436Z)

```

C:\Users\avidu\Projects\badminton-highlight-
indexer\.claude\worktrees\nightly-e2e/scratch/test_ai_segmenter.py
C:\Users\avidu\Projects\badminton-highlight-
indexer\.claude\worktrees\nightly-e2e/tests/test_motion_segmenter.py

```

12. user (2026-06-22T09:08:21.372Z)

No files found

13. user (2026-06-22T09:08:22.401Z)

No files found

14. user (2026-06-22T09:08:23.029Z)

```
C:\Users\avidu\Projects\badminton-highlight-  
indexer\.claude\worktrees\nightly-e2e/backend/pipeline/segmenters/fusion_hybrid.py  
C:\Users\avidu\Projects\badminton-highlight-  
indexer\.claude\worktrees\nightly-e2e/backend/pipeline/segmenters/tracknet_hybrid.py  
C:\Users\avidu\Projects\badminton-highlight-  
indexer\.claude\worktrees\nightly-e2e/backend/pipeline/segmenters/trajectory_hybrid.py  
C:\Users\avidu\Projects\badminton-highlight-  
indexer\.claude\worktrees\nightly-e2e/backend/pipeline/segmenters/wasb_hybrid.py  
C:\Users\avidu\Projects\badminton-highlight-  
indexer\.claude\worktrees\nightly-e2e/backend/pipeline/segmenters/yolo_hybrid.py  
C:\Users\avidu\Projects\badminton-highlight-  
indexer\.claude\worktrees\nightly-e2e/tests/test_fusion_hybrid.py  
C:\Users\avidu\Projects\badminton-highlight-  
indexer\.claude\worktrees\nightly-e2e/tests/test_tracknet_hybrid.py  
C:\Users\avidu\Projects\badminton-highlight-  
indexer\.claude\worktrees\nightly-e2e/tests/test_trajectory_hybrid_equivalence.py  
C:\Users\avidu\Projects\badminton-highlight-  
indexer\.claude\worktrees\nightly-e2e/tests/test_yolo_hybrid.py
```

15. user (2026-06-22T09:08:25.603Z)

```
1      """Shared engine for trajectory-substrate hybrid segmenters.  
2  
3      tracknet_hybrid and wasb_hybrid were copy-paste twins ? CODE_MAP carried a  
4      standing footgun: "any P3/P4 fix must be applied to BOTH". This base class is  
5      now the single home for the whole pipeline shape:  
6  
7          healthcheck gate -> P2 trajectory -> candidate windows (+ persistence)  
8          -> P3 AI-provider handoff over padded clips -> P4 DB population/linking  
9  
10     A concrete subclass supplies only its identity and runner wiring:  
11     - ``family``          ? config section under ``indexing.*``, output subdir, and  
12                           the ``<family>-hybrid-<model>`` detector tag in metadata.  
13     - ``display_label`` ? human label for log lines ("WASB", "TrackNet").  
14     - ``name`` / ``description`` properties (the registry contract).  
15     - ``_build_runner(idx_cfg)`` ? returns (DetectorRunner, detector-specific  
16                                   detection_params extras such as weights_path).  
17  
18     This base is deliberately NOT registered; only concrete subclasses register.  
19     """  
20     import os  
21     import time  
22     import logging  
23     import concurrent.futures  
24     from typing import Any, Dict, List, Optional, Tuple  
25  
26     import cv2  
27  
28     from backend.pipeline.segmenters.base import BasePlaySegmenter  
29     from backend.utils.video import get_video_duration, extract_video_chunk  
30     from backend.utils.paths import output_base  
31     from backend.sports import sport_registry  
32     from backend.providers import provider_registry  
33     from backend.pipeline.detectors.base import DetectorRunner  
34  
35     logger = logging.getLogger(__name__)  
36  
37  
38     def _fmt_ts(seconds: float) -> str:
```

```

39         seconds = max(0, int(seconds))
40         h, rem = divmod(seconds, 3600)
41         m, s = divmod(rem, 60)
42         return f"{h:02d}:{m:02d}:{s:02d}"
43
44
45     class TrajectoryHybridSegmenter(BasePlaySegmenter):
46         """Trajectory-detector hybrid: precise candidate rallies + AI semantic
47         boundaries."""
48         #: config section under `indexing` (velocity_threshold lives there), the
49         #: output subdir for trajectories, and the metadata detector-tag prefix.
50         family: str = ""
51         #: human-readable label used in log lines.
52         display_label: str = ""
53
54         def _build_runner(self, idx_cfg: Dict[str, Any]) -> Tuple[DetectorRunner, Dict[str,
55         Any]]:
56             """Return (runner, detector-specific detection_params extras) for the default
57             (WSL) path."""
58             raise NotImplementedError
59
60         def _build_native_runner(self, idx_cfg: Dict[str, Any]) -> Tuple[DetectorRunner,
61         Dict[str, Any]]:
62             """Return (runner, extras) for the Linux-native (no-WSL) path. Only families
63             with a
64             native runner override this; the base refuses with a clear message (M3.5: WASB
65             only)."""
66             raise NotImplementedError(
67                 f"detector_impl='native' is not supported for the "
68                 f"{self.display_label or self.family or 'trajectory'!r} family ? only WASB
69                 has a "
70                 f"Linux-native runner. Use detector_impl='auto' (WSL) or 'stub'.")
71
72         def _resolve_runner(self, idx_cfg: Dict[str, Any]) -> Tuple[DetectorRunner,
73         Dict[str, Any]]:
74             """Resolve the detector runner, honouring the CPU-stub and Linux-native
75             overrides.
76
77             ``RALLY_DETECTOR_IMPL`` env var (takes precedence) or
78             ``indexing.detector_impl``:
79             ``"stub"`` swaps in a deterministic CPU runner (no GPU/WSL) so the whole
80             pipeline
81             is regression-testable without a GPU ? "mock a GPU with a CPU"; ``"native"``
82             swaps
83             in the Linux-native WASB runner (no WSL/conda-activate; GPU box) via
84             ``_build_native_runner``. Anything else (``"auto"``) delegates to
85             ``_build_runner``
86             (the WSL runner) so the default production path is unchanged
87             (``docs/DECOUPLED_COMPUTE``).
88
89             """
90             impl = (os.environ.get("RALLY_DETECTOR_IMPL") or idx_cfg.get("detector_impl") or
91             "auto").lower()
92             if impl == "stub":
93                 from backend.pipeline.detectors.stub_runner import StubDetectorRunner,
94                 StubConfig
95                 logger.info("%s: detector_impl=stub ? CPU mock runner (no GPU/WSL).",
96                             self.display_label or "trajectory")
97                 return StubDetectorRunner(StubConfig.from_indexing_cfg(idx_cfg)),
98                 {"detector_impl": "stub"}
99             if impl in ("native", "native_wasb", "wasb_native"):
100                 logger.info("%s: detector_impl=native ? Linux-native runner (no WSL).",
101                             self.display_label or "trajectory")
102                 return self._build_native_runner(idx_cfg)
103             return self._build_runner(idx_cfg)

```

```

88
89 @staticmethod
90 def _probe_fps_width(video_path: str) -> tuple:
91     """Return (fps, frame_width) for a video, with sensible fallbacks."""
92     cap = cv2.VideoCapture(video_path)
93     fps = cap.get(cv2.CAP_PROP_FPS) if cap.isOpened() else 0.0
94     width = cap.get(cv2.CAP_PROP_FRAME_WIDTH) if cap.isOpened() else 0.0
95     cap.release()
96     return (fps if fps > 0 else 30.0, width if width > 0 else 1280.0)
97
98 @staticmethod
99 def _shot_count_from(segment: Dict[str, Any], duration: float) -> int:
100     """Provider-reported exchange count, else a duration-based estimate.
101
102     One canonical implementation ? the twins had drifted (wasb relied on
103     ``int(None)`` raising into the fallback; same result, by accident).
104     """
105     shot_count = segment.get("shuttle_exchange_count")
106     if shot_count is None:
107         return max(3, int(duration / 0.8))
108     try:
109         return int(shot_count)
110     except (ValueError, TypeError):
111         return max(3, int(duration / 0.8))
112
113 # --- Fusion seams (identity by default ? wasb_hybrid/tracknet_hybrid unchanged)
114 -----
115 def _enrich_candidate_stats(self, cw: Dict[str, Any], points: List[Any], fps: float,
116                             frame_width: float, video_path: str,
117                             idx_cfg: Dict[str, Any]) -> Dict[str, Any]:
118     """Stats dict persisted into ``candidate_segments.stats`` for one window. The
119     BASE
120     returns exactly the 4 motion keys, so the trajectory twins persist byte-
121     identical
122     rows; ``FusionSegmenter`` overrides this to add ``net_crossings`` (+ person-cue
123     features). ``points`` are the whole-video trajectory points
124     (TrajectoryPoint)."""
125     return {
126         "peak_velocity": cw["peak_velocity"],
127         "mean_velocity": cw["mean_velocity"],
128         "active_frame_count": cw["active_frame_count"],
129         "track_id_count": cw["track_id_count"],
130     }
131
132 def _select_for_handoff(self, windows: List[Dict[str, Any]],
133                         window_stats: List[Dict[str, Any]],
134                         idx_cfg: Dict[str, Any]) -> List[int]:
135     """Indices of the candidate windows that proceed to the AI handoff (and thus may
136     become play_segments). The BASE sends ALL windows (no gating ? twins unchanged);
137     ``FusionSegmenter`` overrides this to apply Gate A/B when thresholds are raised.
138     Persisted candidates are NOT affected ? they remain the full superset for
139     offline
140     re-scoring."""
141     return list(range(len(windows)))
142
143 def process_video(self, video_id: str, video_path: str, # type: ignore[override]
144                  player_pool: Optional[List[str]] = None,
145                  max_frames: Optional[int] = None) -> List[Dict[str, Any]]:
146     # override-ignore: the ABC declares the Result contract (Success|Failure)
147     # but every live segmenter still returns a bare list ? the documented
148     # deliberate-incremental gap (CODE_MAP gotchas; main.py handles both).
149     label = self.display_label
150     # --- Sport profile + AI provider wiring (identical across segmenters) ---
151     sport_name = self.config.get("sport", "badminton")
152     try:

```

```

148         sport_profile = sport_registry.get(sport_name)
149     except KeyError as e:
150         logger.error(str(e))
151         return []
152
153     idx_cfg = self.config.get("indexing", {})
154     # Fully-local, NO-AI mode (default OFF): the trajectory candidate windows become
the
155     # rallies directly ? Phase 3's AI handoff is skipped, so no provider / API key
is
156     # needed and nothing is uploaded. Used to run the local detector standalone
(e.g. to
157     # measure it against the Gemini path, or to avoid the cloud entirely). With
158     # FusionSegmenter the windows are still Gate-A/B-filtered via
`_select_for_handoff`.
159     skip_ai = bool(idx_cfg.get("skip_ai_handoff", False))
160     provider_name = self.config.get("ai_provider", idx_cfg.get("ai_provider",
"gemini"))
161     if skip_ai:
162         model_name = "local-noai"
163         logger.info("%s in FULLY-LOCAL mode (skip_ai_handoff=True): candidate
windows will "
164                     "be emitted as rallies; no %s handoff, no API key needed.",
165                     label, provider_name)
166     else:
167         model_name = self.config.get("ai_model", idx_cfg.get("ai_model",
"gemini-2.5-flash"))
168         api_key_env_var = f"{provider_name.upper()}_API_KEY"
169         api_key = os.environ.get(api_key_env_var)
170         if not api_key:
171             from backend.pipeline.segmenters._keyhint import api_key_hint
172             logger.error(
173                 f"{api_key_env_var} environment variable is not set! "
174                 f"To run the {provider_name} handoff, set your API key. "
175                 + api_key_hint(api_key_env_var)
176             )
177             return []
178         try:
179             provider = provider_registry.get(provider_name, api_key=api_key,
model_name=model_name)
180         except KeyError as e:
181             logger.error(str(e))
182             return []
183
184     video_duration = get_video_duration(video_path)
185     if video_duration <= 0:
186         logger.error("Invalid video duration.")
187         return []
188     fps, frame_width = self._probe_fps_width(video_path)
189
190     # --- Runner config + windowing thresholds ---
191     # _resolve_runner honours the CPU-stub override (RALLY_DETECTOR_IMPL /
192     # indexing.detector_impl="stub") so the pipeline runs GPU/WSL-free; otherwise
193     # it delegates to the subclass _build_runner (the real WSL/native runner).
194     try:
195         runner, runner_params = self._resolve_runner(idx_cfg)
196     except NotImplementedError as e:
197         # e.g. detector_impl="native" for a family without a native runner
(tracknet, or
198         # fusion with a tracknet substrate). Refuse cleanly (no rallies) instead of
crashing.
199         logger.error("%s: %s", label, e)
200         return []
201
202     velocity_thresh = idx_cfg.get(self.family, {}).get("velocity_threshold", 0.02)

```



```

203         merge_gap = idx_cfg.get("dead_time_merge_gap", 2.0)
204         min_window_duration = idx_cfg.get("min_action_window_duration", 2.0)
205         # Bounded-windowing over-merge fix (W1): 0 = OFF (unchanged). See
IndexingConfig.
206         max_window_duration = idx_cfg.get("max_window_duration", 0.0)
207         window_min_split_gap = idx_cfg.get("window_min_split_gap", 0.5)
208         window_hard_cap = idx_cfg.get("window_hard_cap", False)
209         window_inactivity_end = idx_cfg.get("window_inactivity_end", 0.0)
210         inactivity_rally_thresh = idx_cfg.get("inactivity_rally_thresh", 0.08)
211         inactivity_min_density = idx_cfg.get("inactivity_min_density", 0.34)
212         window_blob_fallback = idx_cfg.get("window_blob_fallback", False)
213         window_blob_factor = idx_cfg.get("window_blob_factor", 4.0)
214         handoff_padding = idx_cfg.get("ai_handoff_padding", 2.0)
215         ai_max_workers = idx_cfg.get("ai_max_workers", 5)
216         log_candidates = idx_cfg.get("log_yolo_candidates", True)
217         store_candidates = idx_cfg.get("store_yolo_candidates", True)
218
219         detection_params = {
220             "detector": self.name,
221             "velocity_thresh": velocity_thresh,
222             "merge_gap": merge_gap,
223             "min_action_window_duration": min_window_duration,
224             **runner_params,
225         }
226
227         # --- Phase 1: env healthcheck (fail fast with a clear message) ---
228         ok, msg = runner.healthcheck()
229         if not ok:
230             logger.error("%s detector environment not ready: %s", label, msg)
231             return []
232         logger.info("%s detector env OK: %s", label, msg)
233
234         # --- Phase 2: trajectory -> candidate rally windows ---
235         # Trajectory detectors process the whole video in one pass (they carry
236         # their own frame buffering), so unlike yolo_hybrid we don't pre-chunk.
237         t_start = time.time()
238         out_dir = os.path.join(output_base(self.config), self.family)
239         csv_path = runner.run_predict(video_path, out_dir, video_id=video_id)
240         if not csv_path:
241             logger.error("%s produced no trajectory; aborting.", label)
242             return []
243
244         points = runner.parse_trajectory_csv(csv_path)
245         logger.info("Parsed %d trajectory points (%d visible).",
246                     len(points), sum(1 for p in points if p.visible))
247
248         all_candidate_windows = runner.trajectory_to_action_windows(
249             points, fps=fps, frame_width=frame_width, chunk_start=0.0,
250             velocity_thresh=velocity_thresh, merge_gap=merge_gap,
251             min_window_duration=min_window_duration, chunk_index=0,
252             max_window_duration=max_window_duration, min_split_gap=window_min_split_gap,
253             window_hard_cap=window_hard_cap,
254             window_inactivity_end=window_inactivity_end,
255             inactivity_rally_thresh=inactivity_rally_thresh,
256             inactivity_min_density=inactivity_min_density,
257             window_blob_fallback=window_blob_fallback,
258             window_blob_factor=window_blob_factor,
259         )
260         logger.info("Phase 2 (%s) completed in %.1fs. Candidate windows: %d",
261                     label, time.time() - t_start, len(all_candidate_windows))
262
263         if log_candidates:
264             for cw in all_candidate_windows:
265                 logger.info(f"[{label}] rally]
{_fmt_ts(cw['start'])}-{_fmt_ts(cw['end'])} "

```

```

266             f"({cw['end'] - cw['start']:.1f}s) |
frames={cw['active_frame_count']}] "
267             f"peak_v={cw['peak_velocity']:.3f}
mean_v={cw['mean_velocity']:.3f}")
268
269         # Per-window stats via the enrichment hook ? base = the 4 motion keys; the
fusion
270         # segmenter adds net_crossings + person-cue features. Computed once, reused for
both
271         # persistence and the handoff gate below.
272         window_stats = [
273             self._enrich_candidate_stats(cw, points, fps, frame_width, video_path,
idx_cfg)
274             for cw in all_candidate_windows
275         ]
276
277         # Persist raw candidates for the fine-tuning dataset (same table as YOLO).
278         candidate_ids: List[Optional[int]] = [None] * len(all_candidate_windows)
279         if store_candidates:
280             for i, cw in enumerate(all_candidate_windows):
281                 candidate_ids[i] = self.db.add_candidate_segment(
282                     video_id, detector=self.name,
283                     start_time=cw["start"], end_time=cw["end"],
284                     chunk_index=cw["chunk_index"], confidence=min(1.0,
cw["peak_velocity"]),
285                     stats=window_stats[i], detection_params=detection_params,
286                 )
287
288         if not all_candidate_windows:
289             return []
290
291         # --- Phase 3: AI provider handoff over padded candidate clips ---
292         # Which windows reach the handoff (base = all; fusion may apply Gate A/B).
Persisted
293         # candidates above stay the full superset ? only the served play_segments are
gated.
294         handoff_indices = self._select_for_handoff(all_candidate_windows, window_stats,
idx_cfg)
295
296         ai_segments: List[dict] = []
297         if skip_ai:
298             # Fully-local: the selected candidate windows ARE the rallies ? emit them
verbatim
299             # (no clip extraction, no upload). Boundaries are the local detector's; shot
count
300             # falls back to the duration-based estimate (no AI exchange count).
301             ai_segments = [
302                 {
303                     "start_time": all_candidate_windows[wi]["start"],
304                     "end_time": all_candidate_windows[wi]["end"],
305                     "shuttle_exchange_count": None,
306                     "shot_count_confidence": "LocalNoAI",
307                     "_candidate_id": candidate_ids[wi],
308                 }
309                 for wi in handoff_indices
310             ]
311             logger.info("Phase 3 SKIPPED (fully-local): emitted %d candidate windows as
rallies "
312                         "(no AI handoff).", len(ai_segments))
313         else:
314             handoff_dir = os.path.join(output_base(self.config),
f"chunks_{provider_name}_handoff")
315             os.makedirs(handoff_dir, exist_ok=True)
316             logger.info("Phase 3: %s validation on %d candidate windows...",
provider_name, len(handoff_indices))

```

```

318
319     def process_candidate_window(wi: int, window: Dict[str, Any]) -> List[dict]:
320         w_start = max(0, window["start"] - handoff_padding)
321         w_end = min(video_duration, window["end"] + handoff_padding)
322         w_dur = w_end - w_start
323         w_path = os.path.join(handoff_dir, f"handoff_{video_id}_{wi}.mp4")
324         if not extract_video_chunk(video_path, w_start, w_dur, w_path,
reencode=True):
325             return []
326         prompt = sport_profile.get_prompt(chunk_duration=w_dur)
327         segments, _ = provider.analyze_video(w_path, prompt,
chunk_duration=w_dur,
328                                         label=f"Handoff {wi+1}")
329         valid = []
330         for seg in segments:
331             try:
332                 seg["start_time"] = float(seg["start_time"]) + w_start
333                 seg["end_time"] = float(seg["end_time"]) + w_start
334                 seg["_candidate_id"] = candidate_ids[wi]
335                 valid.append(seg)
336             except (ValueError, TypeError, KeyError) as e:
337                 logger.warning("Dropping segment with unparseable timestamps: %s
? %s", seg, e)
338             try:
339                 os.remove(w_path)
340             except OSError:
341                 pass
342         return valid
343
344     with concurrent.futures.ThreadPoolExecutor(max_workers=ai_max_workers) as
executor:
345         futures = [executor.submit(process_candidate_window, wi,
all_candidate_windows[wi])
346                     for wi in handoff_indices]
347         for future in futures:
348             ai_segments.extend(future.result())
349         try:
350             os.rmdir(handoff_dir)
351         except OSError:
352             pass
353         logger.info("Phase 3 completed. AI confirmed %d semantic play segments.",
len(ai_segments))
354
355     # --- Phase 4: DB population + candidate linking ---
356     segments_data = []
357     for segment in ai_segments:
358         try:
359             start = float(segment["start_time"])
360             end = float(segment["end_time"])
361             dur = end - start
362             metadata = {
363                 "shot_count": self._shot_count_from(segment, dur),
364                 "shot_count_confidence": segment.get("shot_count_confidence",
"Unknown"),
365                 "detector": f"{self.family}-hybrid-{model_name}",
366             }
367             segment_id = self.db.add_play_segment(video_id, start, end, None, None,
metadata)
368             if segment_id:
369                 candidate_id = segment.get("_candidate_id")
370                 if candidate_id is not None:
371                     self.db.link_candidate_to_segment(candidate_id, segment_id)
372                 segments_data.append({
373                     "id": segment_id, "start_time": start, "end_time": end,
374                     "duration": dur, "metadata": metadata,

```

```

375         })
376     except Exception as e:
377         logger.warning("Skipping invalid segment struct: %s", e)
378
379     return segments_data
380

```

16. user (2026-06-22T09:08:27.032Z)

C:\Users\avidu\Projects\badminton-highlight-indexer\.
 claude\worktrees\nightly-e2e/backend/pipeline/stitcher/compiler.py
 C:\Users\avidu\Projects\badminton-highlight-indexer\.
 claude\worktrees\nightly-e2e/backend/pipeline/stitcher/watermark_i18n.py

17. user (2026-06-22T09:08:30.299Z)

```

1     import os
2     import subprocess
3     import tempfile
4     import logging
5     from pathlib import Path
6     from typing import Any, Dict, List, Optional, Tuple, Union
7
8     from backend.utils.ffmpeg import ffmpeg_bin, ffprobe_bin # P2a: single ffmpeg/ffprobe
resolver
9
10    logger = logging.getLogger(__name__)
11
12    # Bundled, commercially-clean default watermark font (Apache-2.0). Shipped so the
overlay
13    # renders out-of-the-box without a system fontconfig setup (a common gap on Windows,
where
14    # `font='Sans'` fails). Overridable via stitching.watermark.font_path. This module lives
at
15    # backend/pipeline/stitcher/, so parents[2] == backend/.
16    _BUNDLED_FONT = Path(__file__).resolve().parents[2] / "assets" / "fonts" / "RobotoSlab-
Regular.ttf"
17
18    class VideoCompiler:
19        def __init__(self, output_dir: str, end_padding: float = 1.0, begin_padding: float =
0.5,
20                    watermark: Optional[Any] = None):
21            self.output_dir = output_dir
22            self.end_padding = end_padding
23            self.begin_padding = begin_padding
24            # Optional branding overlay. Accepts a WatermarkConfig model or a plain dict;
25            # normalized to a dict (or None) so this module stays config-package-agnostic.
26            self.watermark = self._normalize_watermark(watermark)
27            os.makedirs(output_dir, exist_ok=True)
28
29        @staticmethod
30        def _normalize_watermark(watermark: Optional[Any]) -> Optional[Dict[str, Any]]:
31            if watermark is None:
32                return None
33            if hasattr(watermark, "model_dump"):
34                return watermark.model_dump()
35            if isinstance(watermark, dict):
36                return dict(watermark)
37            return None
38
39        @staticmethod
40        def _ff_quote(path: str) -> str:
41            """Make a filesystem path safe to embed inside single quotes in an ffmpeg
42            filtergraph operand (drawtext fontfile/textfile, movie= source): forward
43            slashes (Windows-friendly), escaped single quotes, AND escaped colons.

```

```

44
45     The colon MUST be escaped even inside single quotes: ffmpeg's filtergraph
46     option parser treats ':' as the option separator, so a Windows drive-letter
47     path like ``C:/dir/font.ttf`` otherwise truncates the operand at ``C`` and
48     the encode fails with "Invalid argument". This silently disabled the
49     on-by-default bundled-font watermark (and absolute logo paths) on Windows.""
50     return path.replace("\\", "/").replace("'", r"\'").replace(":", r"\:")
51
52     @staticmethod
53     def _parse_time(val: Union[int, float, str]) -> float:
54         """Normalizes a timestamp value to float seconds.
55         Handles: float/int (pass-through), plain numeric strings ("12.4"),
56         and colon-separated strings ("MM:SS", "HH:MM:SS", "H:MM:SS.ms").
57         This mirrors the parse_time logic used in gemini.py to ensure
58         the stitcher can always consume whatever format the indexer produced."""
59         if isinstance(val, (int, float)):
60             return float(val)
61         if isinstance(val, str):
62             val = val.strip()
63             if ":" in val:
64                 # Handle MM:SS, HH:MM:SS, etc.
65                 parts = val.split(":")
66                 parts.reverse()
67                 total = 0.0
68                 for i, p in enumerate(parts):
69                     try:
70                         total += float(p) * (60 ** i)
71                     except ValueError:
72                         pass
73                 return total
74             try:
75                 return float(val)
76             except ValueError:
77                 logger.warning(f"Could not parse timestamp value '{val}' as a number.
Defaulting to 0.0.")
78                 return 0.0
79                 logger.warning(f"Unexpected timestamp type {type(val).__name__} for value
'{val}'. Defaulting to 0.0.")
80                 return 0.0
81
82     @staticmethod
83     def _merge_overlapping(segments: List[Tuple[float, float]]) -> List[Tuple[float,
float]]:
84         """Normalize, sort, and merge overlapping/duplicate (start, end) ranges.
85
86         A highlight reel must never replay the same footage. The same rally can land in
87         the segment list more than once ? e.g. two detector runs accumulated for one
88         video (add_video is an UPSERT, so re-running without a prune leaves both sets),
89         or
90         chunk-overlap duplicates near boundaries. Stitching those verbatim plays the
91         rally
92         twice. Merging any range that overlaps or abuts the previous one collapses true
93         duplicates/overlaps while preserving genuinely distinct (gapped) rallies.
94         Degenerate rows (end <= start) are dropped. Timestamps may be float/int/str.
95         """
96         parsed = []
97         for raw_s, raw_e in segments:
98             s = VideoCompiler._parse_time(raw_s)
99             e = VideoCompiler._parse_time(raw_e)
100             if e > s:
101                 parsed.append((s, e))
102         parsed.sort()
103         merged: List[Tuple[float, float]] = []
104         for s, e in parsed:
105             if merged and s <= merged[-1][1]: # overlaps/abuts previous -> extend it

```

```

104         merged[-1] = (merged[-1][0], max(merged[-1][1], e))
105     else:
106         merged.append((s, e))
107     return merged
108
109 def _get_video_duration(self, video_path: str) -> float:
110     """Probes the video file to get its total duration in seconds.
111     Returns 0.0 if it cannot be determined."""
112     try:
113         result = subprocess.run(
114             [
115                 ffprobe_bin(), "-v", "error",
116                 "-show_entries", "format=duration",
117                 "-of", "default=noprint_wrappers=1:nokey=1",
118                 video_path
119             ],
120             capture_output=True, text=True, check=True
121         )
122         return float(result.stdout.strip())
123     except Exception as e:
124         logger.warning(f"Could not determine video duration via ffprobe: {e}")
125         return 0.0
126
127 def _watermark_filter(self, tmp_dir: str) -> Optional[str]:
128     """Build the ffmpeg -vf filtergraph that burns the configured watermark (text
129     and/or logo PNG) into a clip during the per-clip re-encode. Returns None when
130     watermarking is disabled or has nothing to draw. The concat stage stays -c copy
131     because the mark is already baked into every clip identically."""
132     wm = self.watermark
133     if not wm or not wm.get("enabled"):
134         return None
135
136     text = (wm.get("text") or "").strip()
137     logo_path = (wm.get("logo_path") or "").strip()
138     if logo_path and not os.path.exists(logo_path):
139         logger.warning(f"[watermark] logo not found: {logo_path} ? skipping logo
140 overlay.")
141     logo_path = ""
142     if not text and not logo_path:
143         return None
144
145     margin = int(wm.get("margin", 24))
146     opacity = float(wm.get("opacity", 0.85))
147     position = str(wm.get("position", "bottom-right"))
148     # drawtext uses lowercase w/h (main) + tw/th (text); overlay uses W/H (main) +
149     w/h (logo).
150     dt_xy = {
151         "bottom-right": f"x=w-tw-{margin}:y=h-th-{margin}",
152         "bottom-left": f"x={margin}:y=h-th-{margin}",
153         "top-right": f"x=w-tw-{margin}:y={margin}",
154         "top-left": f"x={margin}:y={margin}",
155     }.get(position, f"x=w-tw-{margin}:y=h-th-{margin}")
156     ov_xy = {
157         "bottom-right": f"W-w-{margin}:H-h-{margin}",
158         "bottom-left": f"{margin}:H-h-{margin}",
159         "top-right": f"W-w-{margin}:{margin}",
160         "top-left": f"{margin}:{margin}",
161     }.get(position, f"W-w-{margin}:H-h-{margin}")
162
163     drawtext = ""
164     if text:
165         ratio = float(wm.get("font_size_ratio") or 0.035)
166         divisor = max(1, round(1.0 / ratio))
167         # textfile= sidesteps the notorious drawtext text= escaping for arbitrary
168         strings.

```

```

166         tf_path = os.path.join(tmp_dir, "watermark.txt")
167         with open(tf_path, "w", encoding="utf-8") as fh:
168             fh.write(text)
169         # Font precedence: explicit font_path ? bundled Apache-2.0 default (renders
170         # without fontconfig) ? fontconfig family (last-resort, environment-
dependent).
171         font_path = (wm.get("font_path") or "").strip()
172         if not font_path and _BUNDLED_FONT.exists():
173             font_path = str(_BUNDLED_FONT)
174         if font_path:
175             font_opt = f"fontfile='{self._ff_quote(font_path)}'"
176         else:
177             font_opt = f"font='{wm.get('font', 'Sans')}'"
178         box = ""
179         if wm.get("box", True):
180             box = f":box=1:boxcolor=black@{min(opacity, 0.5):.2f}:boxborderw=10"
181         drawtext = (
182             f"drawtext=textfile='{self._ff_quote(tf_path)}':{font_opt}"
183             f":fontcolor=white@{opacity:.2f}:fontsize=h/{divisor}:{dt_xy}{box}"
184         )
185
186         if logo_path and drawtext:
187             return
188         f"movie='{self._ff_quote(logo_path)}'[wm];[in][wm]overlay={ov_xy},{drawtext}"
189         if logo_path:
190             return f"movie='{self._ff_quote(logo_path)}'[wm];[in][wm]overlay={ov_xy}"
191         return drawtext
192
193     def _trim_one_segment(
194         self,
195         idx: int,
196         raw_start: Union[int, float, str],
197         raw_end: Union[int, float, str],
198         segments: List[Tuple[float, float]],
199         video_duration: float,
200         tmp_dir: str,
201         raw_video_path: str,
202         watermark_filter: Optional[str],
203     ) -> Tuple[Optional[str], Optional[Tuple[int, float, float, str]]]:
204         """Trim a single segment into its own clip file (extracted verbatim from the
205         per-segment body of compile_highlights). Returns (clip_path, skip_record): on
206         success the clip path and None; on a skip (invalid/out-of-range/ffmpeg failure)
207         None and the (idx, start, end, reason) record to record in skipped_segments."""
208         # Normalize timestamps to float seconds ? handles float, int,
209         # plain numeric strings, and colon-separated formats like "MM:SS"
210         start = self._parse_time(raw_start)
211         end = self._parse_time(raw_end)
212         segment_label = f"Segment #{idx+1}/{len(segments)} [{start:.2f}s - {end:.2f}s]"
213
214         # Validate the segment times
215         if start < 0:
216             logger.warning(f"{segment_label}: start_time is negative ({start:.2f}s).
Clamping to 0.")
217             start = 0.0
218
219         if start >= end:
220             logger.error(f"{segment_label}: start_time ({start:.2f}s) >= end_time
({end:.2f}s). Skipping invalid segment.")
221             return None, (idx, start, end, "start >= end")
222
223         # Check if segment extends beyond video duration
224         if video_duration > 0:
225             if start >= video_duration:
226                 logger.error(f"{segment_label}: start_time ({start:.2f}s) is beyond the
video duration ({video_duration:.2f}s). ")

```

```

226         f"This segment cannot be extracted ? the video ran out.
Skipping.")
227         return None, (idx, start, end, "start beyond video end")
228
229     if end > video_duration:
230         original_end = end
231         end = video_duration
232         logger.warning(f"{segment_label}: end_time ({original_end:.2f}s) exceeds
video duration ({video_duration:.2f}s). "
233                       f"Clamping end_time to {video_duration:.2f}s. The segment
may be truncated.")
234
235     clip_path = os.path.join(tmp_dir, f"clip_{idx}.mp4")
236
237     # Precise sub-second trim using fast re-encoding to preserve stream headers
238     padded_start = max(0.0, start - self.begin_padding)
239     padded_end = end + self.end_padding
240
241     # Clamp padded_end to video duration if known
242     if video_duration > 0 and padded_end > video_duration:
243         padded_end = video_duration
244         logger.info(f"{segment_label}: Padded end ({end + self.end_padding:.2f}s)
exceeds video duration. "
245                   f"Clamping to {video_duration:.2f}s (end padding partially
applied).")
246
247     trim_duration = padded_end - padded_start
248     logger.info(f"Trimming {segment_label} -> [{padded_start:.3f}s -
{padded_end:.3f}s] (duration: {trim_duration:.2f}s)")
249
250     base_cmd = [
251         ffmpeg_bin(), "-y",
252         "-ss", str(round(padded_start, 3)),
253         "-to", str(round(padded_end, 3)),
254         "-i", raw_video_path,
255         "-c:v", "libx264",
256         "-preset", "superfast",
257         "-crf", "22",
258         "-c:a", "aac",
259         "-b:a", "128k",
260     ]
261     vf_args = ["-vf", watermark_filter] if watermark_filter else []
262
263     # Try with the watermark first; if that encode fails (e.g. a bad font/logo
264     # path), retry once WITHOUT it so a branding misconfig can never nuke the
265     # whole reel. Clips stay codec-identical either way, so concat -c copy holds.
266     attempts = [vf_args, []] if vf_args else [[]]
267     produced = False
268     last_exc: Optional[subprocess.CallledProcessError] = None
269     cmd = base_cmd + [clip_path]
270     for attempt_vf in attempts:
271         cmd = base_cmd + attempt_vf + [clip_path]
272         try:
273             subprocess.run(cmd, capture_output=True, check=True)
274         except subprocess.CallledProcessError as e:
275             last_exc = e
276             if attempt_vf and vf_args:
277                 _err = (e.stderr.decode(errors="replace").strip()[-300:])
278                 if getattr(e, "stderr", None) else ""
279             logger.warning(
280                 f"{segment_label}: watermark encode failed; retrying without it.
"
281                 f"ffmpeg: {_err or '(no stderr captured)'}"
282             )
283         continue

```



```

284         break
285     if os.path.exists(clip_path) and os.path.getsize(clip_path) > 0:
286         produced = True
287         if not attempt_vf and vf_args:
288             logger.warning(f"{segment_label}: encoded WITHOUT the watermark (the
watermark filter failed).")
289         break
290         last_exc = None # ffmpeg succeeded but produced nothing
291         break
292
293     if produced:
294         return clip_path, None
295     elif last_exc is not None:
296         stderr_msg = last_exc.stderr.decode(errors='replace').strip()
297         logger.error(f"{segment_label}: FFmpeg trim failed.")
298         print(f"  Command: {' '.join(cmd)}")
299         print(f"  FFmpeg stderr: {stderr_msg[-500:]}") # Last 500 chars to avoid
flooding
300         return None, (idx, start, end, f"ffmpeg error: {stderr_msg[-200:]}")

```

18. user (2026-06-22T09:08:31.184Z)

C:\Users\avidu\Projects\badminton-highlight-
indexer\.claude\worktrees\nightly-e2e/backend/pipeline/stitcher/compiler.py
C:\Users\avidu\Projects\badminton-highlight-
indexer\.claude\worktrees\nightly-e2e/backend/pipeline/stitcher/watermark_i18n.py

19. user (2026-06-22T09:08:34.554Z)

```

300         return None, (idx, start, end, f"ffmpeg error: {stderr_msg[-200:]}")
301     else:
302         logger.error(f"{segment_label}: FFmpeg exited successfully but output clip
is missing or empty. Skipping.")
303         return None, (idx, start, end, "empty output clip")
304
305     def compile_highlights(self, raw_video_path: str, segments: List[Tuple[float,
float]], output_filename: str) -> str:
306         """
307         Extracts selected segments from a raw video and stitches them together
308         into a seamless highlights file, applying end_padding to prevent abrupt endings.
309         """
310         if not segments:
311             raise ValueError("No highlight segments provided to compile.")
312
313         # Never stitch the same footage twice: collapse overlapping/duplicate ranges
314         # (e.g. accumulated re-run segments or chunk-overlap dupes) before trimming.
315         original_count = len(segments)
316         segments = self._merge_overlapping(segments)
317         if len(segments) < original_count:
318             logger.info("Merged %d overlapping/duplicate segment(s) before stitching: %d
-> %d.",
319                         original_count - len(segments), original_count, len(segments))
320         if not segments:
321             raise ValueError("No valid (start < end) highlight segments after merge.")
322
323         if not os.path.exists(raw_video_path):
324             raise FileNotFoundError(f"[COMPILER ERROR] Source video file not found:
{raw_video_path}")
325
326         # Defense-in-depth: never let the output name escape output_dir (the API also
327         # validates this at the request boundary).
328         output_path = os.path.join(self.output_dir, os.path.basename(output_filename))
329
330         # Probe video duration so we can detect out-of-range segments
331         video_duration = self._get_video_duration(raw_video_path)

```

```

332         if video_duration > 0:
333             logger.info(f"Source video duration: {video_duration:.2f}s")
334         else:
335             logger.warning("Video duration unknown. Cannot validate segment boundaries
336 against video length.")
337
338         # We will create temporary clips and stitch them using FFmpeg concat demuxer
339         temp_clips = []
340         skipped_segments = []
341
342         # Create a temp directory within output_dir
343         with tempfile.TemporaryDirectory(dir=self.output_dir) as tmp_dir:
344             logger.info(f"Trimming {len(segments)} segments
345 (begin_padding={self.begin_padding}s, end_padding={self.end_padding}s)...")
346             # Build the (optional) watermark filtergraph once; it is applied during
347 every
348             # per-clip re-encode so the mark is baked into each clip identically.
349             watermark_filter = self._watermark_filter(tmp_dir)
350             if watermark_filter:
351                 logger.info("[watermark] applying branding overlay to each clip.")
352                 for idx, (raw_start, raw_end) in enumerate(segments):
353                     clip_path, skip_record = self._trim_one_segment(
354                         idx, raw_start, raw_end, segments, video_duration,
355                         tmp_dir, raw_video_path, watermark_filter,
356                     )
357                     if clip_path is not None:
358                         temp_clips.append(clip_path)
359                     if skip_record is not None:
360                         skipped_segments.append(skip_record)
361
362             # Report summary of skipped segments
363             if skipped_segments:
364                 logger.info(f"\n[SUMMARY] {len(skipped_segments)} out of {len(segments)}
365 segments were SKIPPED:")
366                 for seg_idx, seg_start, seg_end, reason in skipped_segments:
367                     logger.info(f" - Segment #{seg_idx+1} [{seg_start:.2f}s -
368 {seg_end:.2f}s]: {reason}")
369                 logger.info("")
370
371             if not temp_clips:
372                 raise RuntimeError(
373                     f"[COMPILER ERROR] Failed to generate ANY valid highlight clips. "
374                     f"All {len(segments)} segments were skipped. "
375                     f"Check the errors above for details. "
376                     f"Common causes: video file is corrupted, segments are out of range,
377 or ffmpeg is not installed."
378                 )
379
380             logger.info(f"Successfully trimmed {len(temp_clips)}/{len(segments)} clips.
381 Preparing to stitch...")
382
383             # Create a list file for FFmpeg concat demuxer
384             list_file_path = os.path.join(tmp_dir, "clips.txt")
385             # encoding pinned: a non-ASCII clip path would crash on Windows (cp1252
386 default);
387             # ffmpeg's concat demuxer reads the list as UTF-8 on every OS.
388             with open(list_file_path, "w", encoding="utf-8") as f:
389                 for clip in temp_clips:
390                     # Escape single quotes in path
391                     safe_path = clip.replace("'", "'\\'")
392                     f.write(f"file '{safe_path}'\n")
393
394             logger.info("Stitching clips together...")
395             # Stitch them together without re-encoding (since they were encoded with
396 identical settings)

```

```

388         concat_cmd = [
389             ffmpeg_bin(), "-y",
390             "-f", "concat",
391             "-safe", "0",
392             "-i", list_file_path,
393             "-c", "copy",
394             output_path
395         ]
396
397     try:
398         subprocess.run(concat_cmd, capture_output=True, check=True)
399
400         # Final verification
401         if not os.path.exists(output_path) or os.path.getsize(output_path) == 0:
402             raise RuntimeError(
403                 f"[COMPILER ERROR] FFmpeg concatenation appeared to succeed, "
404                 f"but output file is missing or empty: {output_path}"
405             )
406
407         output_size_mb = os.path.getsize(output_path) / (1024 * 1024)
408         logger.info(f"Dynamic highlight reel saved: {output_path}
409 ({output_size_mb:.1f} MB)")
410         return output_path
411     except subprocess.CalledProcessError as e:
412         stderr_msg = e.stderr.decode(errors='replace').strip()
413         raise RuntimeError(
414             f"[COMPILER ERROR] FFmpeg concatenation failed.\n"
415             f"Command: {' '.join(concat_cmd)}\n"
416             f"FFmpeg stderr: {stderr_msg[-500:]}"
417         ) from e

```

20. user (2026-06-22T09:08:36.221Z)

No matches found

21. assistant (2026-06-22T09:08:38.278Z)

Now let me check the test files and look for nondeterminism in the whole pipeline:

22. user (2026-06-22T09:08:39.623Z)

```

1      """CLI: regression-test the production tool against golden-set ground truth.
2
3      Obtains ground truth from an annotation provider (a tier), scores the video's
4      stored predictions, and compares to a snapshotted baseline. Exit code 1 on a
5      detected regression so it can gate CI. See docs/GOLDEN_SET_IMPLEMENTATION.md.
6
7      Examples
8      -----
9      Snapshot the current numbers as the baseline (first run / after a real improvement):
10         python run_regression.py --video-id <md5> --tier file --annotations rallies.csv
11         --update-baseline
12
13     Check for a regression (exit 1 if precision/recall dropped beyond tolerance):
14         python run_regression.py --video-id <md5> --tier file --annotations rallies.csv
15
16     """
17     import os
18     import sys
19     import json
20     import argparse
21     import logging
22
23     from backend.infrastructure.database import Database

```

```

22     from backend.config import load_config
23     from backend.annotations import annotation_registry
24     from backend.eval import regression
25
26
27     def _default_db_path() -> str:
28         cfg = load_config()
29         return os.path.join(cfg.output.output_dir, cfg.output.db_name)
30
31
32     def build_parser() -> argparse.ArgumentParser:
33         p = argparse.ArgumentParser(description="Regression-test the tool vs golden-set
ground truth.")
34         p.add_argument("--video-id", required=True, help="Stored video_id (MD5) whose
predictions to score.")
35         p.add_argument("--tier", default="file",
36                         help=f"Annotation provider tier. Available:
{annotation_registry.list_available()}")
37         p.add_argument("--annotations", help="video_ref for the provider (e.g. CSV path for
the 'file' tier).")
38         p.add_argument("--stage", choices=("candidates", "final"), default="final")
39         p.add_argument("--detector", default=None, help="Restrict candidate scoring to one
detector.")
40         p.add_argument("--iou", type=float, default=0.5)
41         p.add_argument("--tolerance", type=float, default=regression.DEFAULT_TOLERANCE)
42         p.add_argument("--db", default=None)
43         p.add_argument("--baseline", default=regression.DEFAULT_BASELINE_PATH,
help="Baseline JSON path.")
44         p.add_argument("--update-baseline", action="store_true",
45                         help="Snapshot the current numbers as the new baseline instead of
checking.")
46         p.add_argument("--allow-missing-baseline", action="store_true",
47                         help="Treat a missing baseline as a pass (exit 0) instead of a gate
error (exit 3).")
48         p.add_argument("--json", dest="json_out", default=None)
49         return p
50
51
52     def main(argv=None) -> int:
53         logging.basicConfig(level=logging.INFO, format="%(levelname)s %(message)s")
54         args = build_parser().parse_args(argv)
55
56         db_path = args.db or _default_db_path()
57         if not os.path.exists(db_path):
58             print(f"ERROR: database not found at {db_path}. Run the pipeline first.",
file=sys.stderr)
59             return 2
60         db = Database(db_path)
61
62         result = regression.regress_with_provider(
63             db, video_id=args.video_id, video_ref=args.annotations, tier=args.tier,
64             stage=args.stage, iou_threshold=args.iou, detector=args.detector,
65             tolerance=args.tolerance, baseline_path=args.baseline,
66         )
67
68         print(regression.format_result(result))
69
70         if args.update_baseline:
71             regression.save_baseline(result.video_id, result.stage, result.precision,
72                                     result.recall, result.f1, path=args.baseline,
73                                     iou_threshold=result.iou_threshold,
74                                     detector=args.detector,
75                                     gt_fingerprint=result.gt_hash)
76             print(f"Baseline updated in {args.baseline}.")

```

```

77         if args.json_out:
78             with open(args.json_out, "w") as f:
79                 json.dump(result.as_dict(), f, indent=2)
80                 print(f"JSON written to {args.json_out}")
81
82         # Exit codes: 1 = regression, 3 = no baseline to compare (gate cannot pass), 0 = ok.
83         if result.regressed:
84             return 1
85         if not result.has_baseline and not args.update_baseline and not
args.allow_missing_baseline:
86             print("ERROR: no baseline to compare against ? the gate cannot verify anything.
"
87
88                 "Snapshot one with --update-baseline (then commit it), or pass "
89                 "--allow-missing-baseline to accept this run.", file=sys.stderr)
90             return 3
91         return 0
92
93     if __name__ == "__main__":
94         raise SystemExit(main())
95

```

23. user (2026-06-22T09:08:41.931Z)

```

.claude\worktrees\nightly-e2e\backend\pipeline\stitcher\compiler.py:24:      # Optional
branding overlay. Accepts a WatermarkConfig model or a plain dict;
.claude\worktrees\nightly-e2e\backend\pipeline\stitcher\compiler.py:25:      # normalized to a
dict (or None) so this module stays config-package-agnostic.
.claude\worktrees\nightly-e2e\backend\pipeline\stitcher\compiler.py:35:      if
isinstance(watermark, dict):
.claude\worktrees\nightly-e2e\backend\pipeline\stitcher\compiler.py:36:      return
dict(watermark)
.claude\worktrees\nightly-e2e\backend\pipeline\validators\blur.py:40:
cap.set(cv2.CAP_PROP_POS_FRAMES, frame_idx)
.claude\worktrees\nightly-e2e\backend\pipeline\validators\base.py:30:      - context: Dynamic
context dict shared across validators (stores shared values like ffprobe metadata).
.claude\worktrees\nightly-e2e\backend\pipeline\stitcher\watermark_i18n.py:16:font gap can never
nuke the reel. Kept config-package-agnostic (operates on a dict).
.claude\worktrees\nightly-e2e\backend\pipeline\stitcher\watermark_i18n.py:48:      """Return a
watermark dict with ``text`` (and ``font_path`` for non-Latin scripts) localized for
.claude\worktrees\nightly-e2e\backend\pipeline\stitcher\watermark_i18n.py:51:      Accepts a
``WatermarkConfig`` model or a plain dict; returns a dict when it localizes (the shape
.claude\worktrees\nightly-e2e\backend\pipeline\stitcher\watermark_i18n.py:60:      elif
isinstance(watermark, dict):
.claude\worktrees\nightly-e2e\backend\pipeline\stitcher\watermark_i18n.py:61:      wm =
dict(watermark)
.claude\worktrees\nightly-e2e\backend\pipeline\validators\relevance.py:58:
cap.set(cv2.CAP_PROP_POS_FRAMES, frame_idx)
.claude\worktrees\nightly-e2e\backend\pipeline\personalization\serving.py:11:      onto the served
fusion config dict, returning the config the served path WOULD use. Falls
.claude\worktrees\nightly-e2e\backend\pipeline\personalization\serving.py:17:
``LogisticRegression`` via ``from_dict`` (None when the user has no active model or it is a
.claude\worktrees\nightly-e2e\backend\pipeline\personalization\serving.py:59:
``FusionConfig`` dict ? e.g. raised ``min_crossings`` / ``min_players`` / flipped
.claude\worktrees\nightly-e2e\backend\pipeline\personalization\serving.py:60:      ``cue_flags``)
onto a COPY of ``served_cfg`` (the baseline served fusion config dict, i.e.
.claude\worktrees\nightly-e2e\backend\pipeline\personalization\serving.py:91:      ``from_dict``)
when the user has an active model that carries one; None when there is no
.claude\worktrees\nightly-e2e\backend\pipeline\personalization\serving.py:93:      wiring) would
feed this model's ``predict_proba`` over the persisted ``window_stats`` ?
.claude\worktrees\nightly-e2e\backend\pipeline\personalization\serving.py:94:      the same math
``experiment.predict`` runs offline. Nothing calls this on the served path
.claude\worktrees\nightly-e2e\backend\pipeline\personalization\serving.py:103:      return
LogisticRegression.from_dict(blob)
.claude\worktrees\nightly-e2e\backend\pipeline\detectors\base.py:15:-

```

```

run_predict(video_win_path: str, output_win_dir: str) -> Optional[str]
.claude\worktrees\nightly-e2e\backend\pipeline\detectors\base.py:17: (e.g.
".../clip_predict.csv" or ".../clip_wasb.csv"), or None on any failure.
.claude\worktrees\nightly-e2e\backend\pipeline\detectors\base.py:22: Return (ok, message).
Never raises. "ok" decides whether to proceed to run_predict.
.claude\worktrees\nightly-e2e\backend\pipeline\detectors\base.py:31:2. Implement run_predict and
healthcheck (return (bool, str) tuple).
.claude\worktrees\nightly-e2e\backend\pipeline\detectors\base.py:168:     def run_predict(self,
video_win_path: str, output_win_dir: str,
.claude\worktrees\nightly-e2e\backend\pipeline\detectors\base.py:187:     # unconditionally after
run_predict. Concrete runners reuse/re-export the
.claude\worktrees\nightly-e2e\backend\pipeline\validators\scene_cut.py:49:
cap.set(cv2.CAP_PROP_POS_FRAMES, frame_idx)
.claude\worktrees\nightly-e2e\backend\pipeline\segmenters\motion.py:3:import random
.claude\worktrees\nightly-e2e\backend\pipeline\segmenters\motion.py:68:         t_start =
time.time()
.claude\worktrees\nightly-e2e\backend\pipeline\segmenters\motion.py:114:                                     elapsed
= time.time() - t_start
.claude\worktrees\nightly-e2e\backend\pipeline\segmenters\motion.py:181:                                     For each
detected segment this assigns a random server/receiver, fabricates
.claude\worktrees\nightly-e2e\backend\pipeline\segmenters\motion.py:183:                                     with the stdlib
`random` module (the legacy demo's fabricated baseline).
.claude\worktrees\nightly-e2e\backend\pipeline\segmenters\motion.py:184:                                     Returns the per-
segment dicts handed back to the caller. The `random`
.claude\worktrees\nightly-e2e\backend\pipeline\segmenters\motion.py:190:                                     # Assign
Server and Receiver randomly from pool
.claude\worktrees\nightly-e2e\backend\pipeline\segmenters\motion.py:191:                                     server,
receiver = random.sample(player_pool, 2)
.claude\worktrees\nightly-e2e\backend\pipeline\segmenters\motion.py:196:
"shot_count": random.randint(4, 25),
.claude\worktrees\nightly-e2e\backend\pipeline\segmenters\motion.py:198:
"avg_speed_kmh": round(random.uniform(40, 85), 1)
.claude\worktrees\nightly-e2e\backend\pipeline\segmenters\motion.py:206:
serve_foul = random.random() < 0.15 # 15% chance of serve foul
.claude\worktrees\nightly-e2e\backend\pipeline\segmenters\motion.py:212:
details={"status": "foul" if serve_foul else "valid", "type": "low_serve" if random.random() >
0.5 else "high_serve"}
.claude\worktrees\nightly-e2e\backend\pipeline\segmenters\motion.py:215:                                     # B:
Smash Events (1 to 3 random smashes per segment)
.claude\worktrees\nightly-e2e\backend\pipeline\segmenters\motion.py:216:
smashes_count = random.randint(0, 3)
.claude\worktrees\nightly-e2e\backend\pipeline\segmenters\motion.py:218:
smash_time = random.uniform(start + 1.5, end - 1.0)
.claude\worktrees\nightly-e2e\backend\pipeline\segmenters\motion.py:219:
smasher = random.choice([server, receiver])
.claude\worktrees\nightly-e2e\backend\pipeline\segmenters\motion.py:225:
details={"speed_kmh": round(random.uniform(120, 250), 1)}
.claude\worktrees\nightly-e2e\backend\pipeline\segmenters\motion.py:230:                                     is_foul
= random.random() > 0.4
.claude\worktrees\nightly-e2e\backend\pipeline\segmenters\motion.py:231:
error_player = random.choice([server, receiver])
.claude\worktrees\nightly-e2e\backend\pipeline\segmenters\motion.py:238:
details={"type": random.choice(foul_types)} if is_foul else {"shot_type": "drop" if
random.random() > 0.5 else "smash"}
.claude\worktrees\nightly-e2e\backend\pipeline\segmenters\fusion_hybrid.py:4:TrackNet optional)
to seed candidate windows exactly as `wasb_hybrid`/`tracknet_hybrid`
.claude\worktrees\nightly-e2e\backend\pipeline\segmenters\fusion_hybrid.py:6:shuttle-seeded
window via two overridable seams the engine exposes:
.claude\worktrees\nightly-e2e\backend\pipeline\segmenters\fusion_hybrid.py:23:nothing is gated
(thresholds 0, person OFF), so `FusionSegmenter` with default config seeds

```

[Showing results with pagination = limit: 50]

Let me search for more specific determinism issues:

25. user (2026-06-22T09:08:47.242Z)

```
16:The GPU detector pass (one forward per sliding window ? ~21k for a 6-min clip) is
24:  most ``--flush-every`` windows of GPU work instead of the whole pass.
29:  not the GPU hour.
56:def _build_cfg(weights: str, sport: str, device: str = "cuda"):
57:  """Compose the WASB eval config via Hydra, overriding model/weights/device.
59:  ``device`` is ``cuda`` (default ? the historical WSL behaviour, byte-parity preserved),
60:  ``mps`` (Apple) or ``cpu``. Only the single-GPU CUDA path requests ``runner.gpus=[0]``;
61:  cpu/mps run with no GPU list so the detector places tensors on ``device``.
64:  gpus = "[0]" if device == "cuda" else "[]" # single GPU on cuda; none for cpu/mps
70:      f"runner.device={device}",
71:      f"runner.gpus={gpus}", # default config assumes multiple GPUs; pin to this box
138:      frames_in: int, frames_total: int, device: str = "cuda") -> bool:
141:  ``device`` defaults to ``cuda`` so caches written before device-keying (no ``device``
142:  field) stay compatible with a CUDA run ? but a cpu/mps run will NOT reuse a cuda cache
143:  (detections can differ numerically across devices), and vice-versa.
153:      and manifest.get("device", "cuda") == device
235:  """Sliding windows of ``frames_in`` consecutive frame *paths* (the unit of GPU work
251:  ``num_workers=0``. ``ImageDataset.__getitem__(i)`` reads the frames of window ``i``
372:      limit: int = 0, batch_size: int = 8, num_workers: int = 4,
376:      device: str = "cuda") -> str:
389:  ``cv2.cvtColor(bgr, BGR2RGB)`` , verified byte-identical). Streaming forces
``num_workers=0``
390:  (the in-process shim + buffer can't cross worker processes).
407:  cfg = _build_cfg(weights, sport, device)
427:  # Sliding windows of ``frames_in`` consecutive frames (the unit of GPU work + checkpoint).
443:      frames_in=frames_in, frames_total=len(frames), device=device,
461:      "device": device,
478:      # cv2.imread shim live in THIS process and can't be pickled to DataLoader workers.
479:      loader_workers = 0 if streaming else num_workers
480:      loader = DataLoader(ds, batch_size=batch_size, shuffle=False,
num_workers=loader_workers)
492:      f"(total {windows_total}, batch={batch_size}, workers={loader_workers},
"
```

[Showing results with pagination = limit: 30]

26. user (2026-06-22T09:08:48.108Z)

```
27:  is skipped entirely and we just re-run the (cheap, deterministic) tracker over
497:      stack.enter_context(torch.no_grad())
546:  # ---- tracker re-run over the full ordered cache (cheap, deterministic) ---- #
```

27. user (2026-06-22T09:08:52.669Z)

```
250             except (KeyError, ValueError):
251                 continue
252             points.sort(key=lambda p: p.frame)
253             return points
254
255     @staticmethod
256     def _active_frames(points: List[TrajectoryPoint], fps: float, frame_width: float,
257                       velocity_thresh: float) -> List[Tuple[int, float]]:
258         """Per-frame ``(frame_idx, velocity)`` for frames where the shuttle is in
flight.
259
260         Computes each visible point's normalised velocity from the last *visible* point;
261         an absent shuttle breaks the trajectory. A frame qualifies when its velocity
exceeds
262         ``velocity_thresh``. ``fps``/``frame_width`` are expected to be already
defaulted by
```

```

263         the caller so identical floats feed the velocity math."""
264         active = [] # (frame_idx, velocity)
265         prev: Optional[TrajectoryPoint] = None
266         for p in points:
267             if not p.visible:
268                 prev = None # break the trajectory; absent shuttle = not in flight
269                 continue
270             if prev is not None:
271                 dframes = p.frame - prev.frame
272                 if dframes > 0:
273                     dist = ((p.x - prev.x) ** 2 + (p.y - prev.y) ** 2) ** 0.5
274                     velocity = (dist / frame_width) * (fps / dframes)
275                     if velocity > velocity_thresh:
276                         active.append((p.frame, velocity))
277             prev = p
278         return active
279
280     @staticmethod
281     def trajectory_to_action_windows(
282         points: List[TrajectoryPoint],
283         fps: float,
284         frame_width: float,
285         chunk_start: float = 0.0,
286         velocity_thresh: float = 0.02,
287         merge_gap: float = 2.0,
288         min_window_duration: float = 2.0,
289         chunk_index: Optional[int] = None,
290         max_window_duration: float = 0.0,
291         min_split_gap: float = 0.5,
292         window_hard_cap: bool = False,
293         window_inactivity_end: float = 0.0,
294         inactivity_min_density: float = 0.34,
295         inactivity_rally_thresh: float = 0.08,
296         window_blob_fallback: bool = False,
297         window_blob_factor: float = 4.0,
298     ) -> List[Dict[str, Any]]:
299         """Convert a shuttle trajectory into candidate rally windows.
300
301         A frame is "active" when the shuttle is visible AND its inter-frame
302         displacement (normalised by frame width, per second) exceeds
303         ``velocity_thresh`` ? i.e. the shuttle is in flight. Active frames within
304         ``merge_gap`` seconds of each other are grouped into one window; short
305         windows are dropped. Mirrors HybridYoloSegmenter's windowing so the dict
306         shape feeding the AI-handoff phase is identical.
307
308         ``max_window_duration`` (0 = OFF, the default ? behaviour unchanged): a guard
309         against the *over-merge* failure where "shuttle moving" is conflated with "rally in
310         progress" and a single window swallows several rallies + the dead time between them. When
311         set, any window longer than this is recursively SPLIT at its largest internal inactivity
312         gap (the longest stretch with no in-flight shuttle ? the most likely rally boundary),
313         provided that gap is at least ``min_split_gap`` seconds. This is the bounded-windowing
314         fix from ``docs/RALLY_QUALITY_RESEARCH.md`` §6 (W1); it never merges, only cuts, so
315         recall cannot drop, and it is config-gated default-OFF until measured on the golden set.
316
317         ``window_hard_cap`` (default OFF) makes ``max_window_duration`` a TRUE cap: when
318         an over-long window has NO internal inactivity gap to split on (a continuously-
319         active

```



```

319         trajectory ? the shuttle never stops moving, e.g. the real-footage
``mahadevpura-1``
320         blob where the gap-splitter alone left a single 174 s window), it is recursively
321         hard-chopped at its temporal midpoint until every piece is ? the cap. Still only
cuts
322         (never merges), so recall cannot drop; gated default-OFF until measured.
323
324         ``window_blob_fallback`` (default OFF ? the R5 last-resort blob splitter):
unlike
325         ``window_hard_cap`` (which midpoint-chops BEFORE the R4 trim, so it fires on
every gap-less
326         over-cap window and over-segments normal clips), this runs AFTER the R4
inactivity trim and
327         force-chops ONLY a genuine *blob* ? a group still longer than
``window_blob_factor`` x
328         ``max_window_duration`` once the gap-split and the principled rally-end trim
have both had
329         their chance (e.g. mahadevpura-1's ~65 s residual ? 11x a 6 s cap; a 7 s rally
is left
330         alone). The blob is midpoint-chopped down to ? the cap, those forced cuts are
logged, and
331         each resulting window is flagged ``forced_cut=True`` ? surfacing the clip as one
that needs
332         rally-STATE cues (net-crossings / two-sided motion), not a bigger cap. Only cuts
(recall
333         can't drop). Requires ``max_window_duration`` > 0; gated default-OFF until
measured.
334         """
335         if fps <= 0:
336             fps = 30.0
337         if frame_width <= 0:
338             frame_width = 1280.0
339
340         active = TrackNetRunner._active_frames(points, fps, frame_width,
velocity_thresh)
341
342         if not active:
343             return []
344
345         max_gap_frames = int(merge_gap * fps)
346
347         def _finalize(group: List[Tuple[int, float]]) -> Dict[str, Any]:
348             vels = [v for _, v in group]
349             return {

```

28. user (2026-06-22T09:08:53.710Z)

```

8:     2. Invoke ``src/predict.py`` inside the TrackNetV4 conda env via ``wsl``.
32:     """Resolved settings for a TrackNet WSL run (built from config.json ->
indexing.tracknet)."""
88:     """Runs TrackNet predict.py in WSL and turns the output CSV into action windows.
120:     def run_predict(self, video_win_path: str, output_win_dir: str,
122:         """Run predict.py on a video. Returns the Windows path to the produced CSV, or None.
127:         therefore predict.py's ``<stem>_predict.csv`` output ? so two videos that share
132:         "TrackNet weights_path is not set. Set indexing.tracknet.weights_path "
139:         # predict.py cwd instead of the repo (same bug class as wasb_runner).
148:         # predict.py only accepts .avi/.mp4 and names the CSV "<stem>_predict.csv".
150:         logger.error("TrackNet predict.py only supports .mp4/.avi (got %s)", ext)
155:         # predict.py derives its output name from the (staged) video stem, so staging
156:         # under the namespaced name propagates into "<key>_predict.csv".
158:         expected_csv_win = os.path.join(output_win_dir, f"{key}_predict.csv")
167:         # No staging: predict.py reads /mnt directly and writes "<stem>_predict.csv".
170:         expected_csv_win = os.path.join(output_win_dir, f"{stem}_predict.csv")
182:         f"python predict.py "
196:         logger.error("TrackNet predict.py failed:\n%s", (res.stderr or

```

```

res.stdout)[-2000:])
208:             "(set indexing.tracknet.keep_frames=true to retain).", key)
238:         """Parse a TrackNet predict CSV (columns: Frame,Visibility,X,Y)."""
305:     windows are dropped. Mirrors HybridYoloSegmenter's windowing so the dict
310:     and a single window swallows several rallies + the dead time between them. When set,
any
315:     drop, and it is config-gated default-OFF until measured on the golden set.
446:     rally) UNLESS ``hard_cap`` is set, in which case such a group is hard-chopped at its

```

29. assistant (2026-06-22T09:08:55.785Z)

Let me check for float threshold ties and other sources of nondeterminism:

30. user (2026-06-22T09:08:57.840Z)

```

<tool_use_error>InputValidationError: Read failed due to the following issue:
The parameter `offset` type is expected as `number` but provided as `array`</tool_use_error>

```

31. user (2026-06-22T09:08:58.867Z)

```

<tool_use_error>InputValidationError: Read failed due to the following issue:
The parameter `offset` type is expected as `number` but provided as `array`</tool_use_error>

```

32. user (2026-06-22T09:09:02.139Z)

```

347     def _finalize(group: List[Tuple[int, float]]) -> Dict[str, Any]:
348         vels = [v for _, v in group]
349         return {
350             "start": group[0][0] / fps + chunk_start,
351             "end": group[-1][0] / fps + chunk_start,
352             "active_frame_count": len(group),
353             "peak_velocity": max(vels),
354             "mean_velocity": sum(vels) / len(vels),
355             "track_id_count": 1, # single shuttle; kept for window-shape parity
356             "chunk_index": chunk_index,
357         }
358
359     groups: List[List[Tuple[int, float]]] = []
360     group = [active[0]]
361     for af in active[1:]:
362         if af[0] - group[-1][0] <= max_gap_frames:
363             group.append(af)
364         else:
365             groups.append(group)
366             group = [af]
367     groups.append(group)
368
369     if max_window_duration > 0:
370         groups = TrackNetRunner._split_overlong_groups(
371             groups, int(max_window_duration * fps), int(min_split_gap * fps),
372             hard_cap=window_hard_cap)
373
374     # R4 ? rally-END inactivity trim (default OFF). Applied AFTER the cap split so
each rally
375     # piece has its own post-rally drift tail removed: the velocity windowing keeps
a window
376     # "active" while the shuttle drifts/gets-picked-up above threshold, over-
extending the END
377     # (the measured gap vs golden). Trim back to the last frame whose trailing
window is still
378     # dense with motion (rally on); a sustained low-density run = rally over. Only
cuts.
379     if window_inactivity_end > 0:
380         tw = int(window_inactivity_end * fps)

```

```

381         groups = [TrackNetRunner._trim_inactive_tail(g, tw, inactivity_min_density,
382                                                         inactivity_rally_thresh) for g
in groups]
383
384         # R5 ? blob-fallback (default OFF). LAST resort: after the gap-split AND the R4
trim, any
385         # group still longer than the cap is a gap-less, rally-end-signal-less blob;
force-chop it to
386         # ? the cap at the midpoint and flag every piece, so the degenerate single-
window outcome is
387         # avoided and the clip is surfaced (logged) as needing rally-STATE cues, not a
bigger cap.
388         forced_flags = [False] * len(groups)
389         if window_blob_fallback and max_window_duration > 0:
390             groups, forced_flags = TrackNetRunner._blob_fallback_chop(
391                 groups, int(max_window_duration * fps), window_blob_factor)
392
393         windows = []
394         for g, forced in zip(groups, forced_flags):
395             w = _finalize(g)
396             if forced:
397                 w["forced_cut"] = True
398             windows.append(w)
399         return [w for w in windows if (w["end"] - w["start"]) >= min_window_duration]
400
401     @staticmethod
402     def _trim_inactive_tail(group: List[Tuple[int, float]], trail_frames: int,
403                             min_density: float, rally_thresh: float) -> List[Tuple[int,
float]]:
404         """R4: trim the post-rally drift tail. After a rally the shuttle keeps moving
*slowly*
405         (picked up / walked / knock-up) above ``velocity_thresh``, so the velocity
windowing
406         over-extends the END. A frame is "fast" (real rally motion) iff its velocity >
``rally_thresh``
407         (higher than the active threshold). The rally is "on" at frame ``i`` while its
trailing
408         ``trail_frames`` window holds >= ``min_density`` fraction of fast frames; trim
back to the
409         LAST such frame ? everything after is sustained slow drift (rally over). Only
cuts (recall
410         can't rise); a window whose tail never goes slow is untouched, and if no fast-
dense point
411         exists at all the group is kept whole (fail-safe, no over-trim). O(n) two-
pointer; the START
412         is left alone (already ~Gemini-accurate per the n=2 boundary-error finding)."""
413         if trail_frames <= 0 or len(group) < 2:
414             return group
415         frames = [f for f, _ in group]
416         fast = [1 if v > rally_thresh else 0 for _, v in group]
417         lo = 0
418         fast_sum = 0
419         last_dense = -1
420         for i in range(len(frames)):
421             fast_sum += fast[i]
422             while frames[lo] <= frames[i] - trail_frames:
423                 fast_sum -= fast[lo]
424                 lo += 1
425             win_count = i - lo + 1
426             if win_count > 0 and fast_sum / win_count >= min_density:
427                 last_dense = i
428         return group[: last_dense + 1] if last_dense >= 0 else group
429
430     @staticmethod
431     def _midpoint_split_index(group: List[Tuple[int, float]]) -> int:

```

```

432         """Index of the active frame nearest the group's temporal midpoint (used to chop
a
433         gap-less group into two). Shared by the W1 hard-cap fallback and the R5 blob-
fallback."""
434         mid = (group[0][0] + group[-1][0]) / 2.0
435         split_i = min(range(1, len(group)), key=lambda i: abs(group[i][0] - mid))
436         return split_i
437
438     @staticmethod
439     def _split_overlong_groups(groups: List[List[Tuple[int, float]]], max_win_frames:
int,
440                               min_split_frames: int,
441                               hard_cap: bool = False) -> List[List[Tuple[int, float]]]:
442         """Recursively split any active-frame group longer than ``max_win_frames`` at
its largest
443         internal gap (? ``min_split_frames``) ? an inactivity-aware cut at the most
likely rally
444         boundary. Splitting only subdivides existing groups (never merges/extends), so
it cannot
445         invent or drop coverage; a group with no gap ? the floor is left intact (a
genuinely long
446         rally) UNLESS ``hard_cap`` is set, in which case such a group is hard-chopped at
its
447         temporal midpoint until every piece is ? ``max_win_frames`` (makes the cap a
true upper
448         bound on continuously-active trajectories). Iterative worklist to bound stack
depth."""
449         if max_win_frames <= 0:
450             return groups
451         out: List[List[Tuple[int, float]]] = []
452         work = list(groups)
453         while work:
454             g = work.pop()
455             if len(g) < 2 or (g[-1][0] - g[0][0]) <= max_win_frames:
456                 out.append(g)
457                 continue
458             best_i, best_gap = -1, -1
459             for i in range(1, len(g)):
460                 gap = g[i][0] - g[i - 1][0]
461                 if gap > best_gap:
462                     best_gap, best_i = gap, i
463             if best_i < 0 or best_gap < min_split_frames:
464                 # Too long but no real dead-time gap. Default: keep as one (a genuine
long rally).
465                 # hard_cap: chop at the temporal midpoint so the cap is actually
enforced.
466                 if hard_cap:
467                     split_i = TrackNetRunner._midpoint_split_index(g)
468                     work.append(g[:split_i])
469                     work.append(g[split_i:])
470                 else:
471                     out.append(g)
472                     continue
473                 work.append(g[:best_i])
474                 work.append(g[best_i:])
475             out.sort(key=lambda grp: grp[0][0])
476             return out
477
478     @staticmethod
479     def _blob_fallback_chop(groups: List[List[Tuple[int, float]]], max_win_frames: int,
480                             blob_factor: float = 4.0
481                             ) -> Tuple[List[List[Tuple[int, float]]], List[bool]]:
482         """R5 blob splitter ? the gap-less, R4-survived residual. After the W1 gap-split
AND the R4
483         inactivity trim have each had their chance, a group still longer than

```

```

``blob_factor`` x
484     ``max_win_frames`` is a genuine continuously-active blob (no internal gap, no
rally-end
485     signal the principled rules could find ? the real-footage ``mahadevpura-1`` ~65
s residual ?
486     11x a 6 s cap). The factor is what keeps this SURGICAL: a normal long rally
(~1?2x the cap)
487     is left ALONE; only the degenerate blob qualifies. As a LAST RESORT the blob is
488     midpoint-chopped to ? the cap and every resulting piece is FLAGGED as a forced
cut, so the
489     degenerate single-window outcome is avoided and the clip is surfaced (logged) as
needing
490     rally-STATE cues ? net-crossings / two-sided motion ? not a bigger cap. Only
cuts (recall
491     can't drop). Returns ``(groups, forced_flags)`` aligned 1:1; ``forced_flags[i]``
is True iff
492     that piece came from a forced chop. Iterative worklist to bound stack depth.""
493     if max_win_frames <= 0:
494         return groups, [False] * len(groups)
495     blob_frames = max_win_frames * max(1.0, blob_factor)
496     out: List[List[Tuple[int, float]]] = []
497     flags: List[bool] = []
498     forced = 0
499     for g in groups:
500         if len(g) < 2 or (g[-1][0] - g[0][0]) <= blob_frames:
501             out.append(g)
502             flags.append(False)
503             continue
504     pieces: List[List[Tuple[int, float]]] = []
505     work = [g]
506     while work:
507         h = work.pop()
508         if len(h) < 2 or (h[-1][0] - h[0][0]) <= max_win_frames:
509             pieces.append(h)
510             continue
511         split_i = TrackNetRunner._midpoint_split_index(h)
512         work.append(h[:split_i])
513         work.append(h[split_i:])
514         pieces.sort(key=lambda grp: grp[0][0])
515         out.extend(pieces)
516         flags.extend([True] * len(pieces))
517         forced += len(pieces) - 1
518     if forced:
519         logger.warning(
520             "R5 blob-fallback: forced %d midpoint cut(s) on continuously-active
group(s) with "
521             "no rally-end signal ? this clip needs rally-STATE cues (net-crossings /
two-sided "
522             "motion), not a bigger cap", forced)
523     return out, flags
524

```

33. assistant (2026-06-22T09:09:04.577Z)

Now let me check the config and understand what golden videos exist:

34. user (2026-06-22T09:09:06.004Z)

```

1      {
2      "sport": "badminton",
3      "ai_provider": "gemini",
4      "ai_model": "gemini-flash-latest",
5      "validation": {
6      "min_resolution_width": 1280,
7      "min_resolution_height": 720,

```

```

8         "min_fps": 29.9,
9         "min_blur_threshold": 20.0,
10        "max_scene_cuts_per_minute": 0.5,
11        "relevance": {
12            "court_green_lower": [
13                35,
14                40,
15                40
16            ],
17            "court_green_upper": [
18                85,
19                255,
20                255
21            ],
22            "court_pixels_min_ratio": 0.05
23        }
24    },
25    "indexing": {
26        "default_segmenter": "gemini",
27        "use_gpu": true,
28        "motion_threshold": 0.08,
29        "min_segment_duration": 3.0,
30        "dead_time_merge_gap": 2.0,
31        "chunk_duration": 90,
32        "chunk_overlap": 10,
33        "detector_model": "fasterrcnn_resnet50_fpn",
34        "detector_score_threshold": 0.5,
35        "yolo_velocity_threshold": 0.15,
36        "yolo_frame_skip": 3,
37        "yolo_tracker": "bytetrack.yaml",
38        "yolo_prefetch_workers": 2,
39        "min_action_window_duration": 2.0,
40        "log_yolo_candidates": true,
41        "store_yolo_candidates": true,
42        "ai_handoff_padding": 2.0,
43        "ai_max_workers": 1,
44        "tracknet": {
45            "wsl_distro": "Ubuntu",
46            "repo_dir": "~/models/TrackNetV4",
47            "conda_sh": "~/miniconda3/etc/profile.d/conda.sh",
48            "conda_env": "TrackNetV4",
49            "weights_path": "",
50            "queue_length": 5,
51            "stage_in_wsl": true,
52            "wsl_stage_dir": "~/clips",
53            "timeout_sec": 1800,
54            "velocity_threshold": 0.02
55        },
56        "wasb": {
57            "wsl_distro": "Ubuntu",
58            "repo_dir": "~/models/WASB-SBDT",
59            "conda_sh": "~/miniconda3/etc/profile.d/conda.sh",
60            "conda_env": "wasb",
61            "weights_path": "~/models/WASB-
62            SBDT/pretrained_weights/wasb_badminton_best.pth.tar",
63            "sport": "badminton",
64            "wsl_stage_dir": "~/clips",
65            "timeout_sec": 1800,
66            "velocity_threshold": 0.02
67        }
68    },
69    "output": {
70        "default_highlights_name": "highlights.mp4",
71        "db_name": "sports_indexer.db"
72    }
73 }

```

```

72     "stitching": {
73         "begin_padding": 0.5,
74         "end_padding": 0.5,
75         "use_cache": false,
76         "min_shot_count": 1,
77         "watermark": {
78             "enabled": true,
79             "text": "Generated by Khelsutra.guru",
80             "position": "bottom-right",
81             "opacity": 0.85,
82             "logo_path": "",
83             "font_path": ""
84         }
85     }
86 }

```

35. user (2026-06-22T09:09:07.360Z)

```

C:\Users\avidu\Projects\badminton-highlight-
indexer\.claude\worktrees\nightly-e2e/backend/eval/fusion_golden.py
C:\Users\avidu\Projects\badminton-highlight-
indexer\.claude\worktrees\nightly-e2e/backend/eval/golden_fixtures.py
C:\Users\avidu\Projects\badminton-highlight-
indexer\.claude\worktrees\nightly-e2e/backend/eval/golden_real_fixtures.py
C:\Users\avidu\Projects\badminton-highlight-
indexer\.claude\worktrees\nightly-e2e/backend/eval/per_user_regression.py
C:\Users\avidu\Projects\badminton-highlight-
indexer\.claude\worktrees\nightly-e2e/backend/eval/regression.py
C:\Users\avidu\Projects\badminton-highlight-
indexer\.claude\worktrees\nightly-e2e/eval_baselines\nightly_baseline.json
C:\Users\avidu\Projects\badminton-highlight-
indexer\.claude\worktrees\nightly-e2e/run_regression.py
C:\Users\avidu\Projects\badminton-highlight-
indexer\.claude\worktrees\nightly-e2e/scripts\nightly_regression.ps1
C:\Users\avidu\Projects\badminton-highlight-
indexer\.claude\worktrees\nightly-e2e/scripts\nightly_regression.py
C:\Users\avidu\Projects\badminton-highlight-
indexer\.claude\worktrees\nightly-e2e/scripts/register_nightly_task.ps1
C:\Users\avidu\Projects\badminton-highlight-
indexer\.claude\worktrees\nightly-e2e/tests/test_eval_regression.py
C:\Users\avidu\Projects\badminton-highlight-
indexer\.claude\worktrees\nightly-e2e/tests/test_fusion_golden_telemetry.py
C:\Users\avidu\Projects\badminton-highlight-
indexer\.claude\worktrees\nightly-e2e/tests/test_golden_fixtures.py
C:\Users\avidu\Projects\badminton-highlight-
indexer\.claude\worktrees\nightly-e2e/tests/test_golden_fixtures_split.py
C:\Users\avidu\Projects\badminton-highlight-
indexer\.claude\worktrees\nightly-e2e/tests/test_golden_real_fixtures.py
C:\Users\avidu\Projects\badminton-highlight-
indexer\.claude\worktrees\nightly-e2e/tests/test_nightly_regression.py
C:\Users\avidu\Projects\badminton-highlight-
indexer\.claude\worktrees\nightly-e2e/tests/test_per_user_regression.py
C:\Users\avidu\Projects\badminton-highlight-
indexer\.claude\worktrees\nightly-e2e/tests/test_served_gate_a_regression.py

```

36. user (2026-06-22T09:09:10.931Z)

```

1     """Nightly rally-quality regression tripwire.
2
3     Re-scores the **golden corpus** under HEAD via :mod:`backend.eval.rally_seg_eval` (CPU,
4     seconds ? the WASB GPU trajectories are CACHED, only the windowing/scorer changes) and
5     diffs
6     the headline metrics against a FROZEN baseline
7     (`eval_baselines/nightly_baseline.json`).
8     Writes a dated report and exits non-zero on any regression beyond tolerance OR any per-

```

```

video
7      over-segmentation-guard regression.
8
9      This is the owner's dual-eval-set discipline made automatic (`memory/eval-dual-set-
regression`)
10     + ``docs/R2_EVAL_METRIC_DESIGN.md`` §4.5): the growing GOLDEN-finalized set is re-run
every
11     night so a *late* change cannot silently lose an *earlier* win. It is **default-
ADDITIVE** ? it
12     does NOT change the promotion gate. It is a *tripwire*, not a re-training run: no GPU,
no Gemini,
13     no WASB re-extraction (those are cached and unchanged night to night).
14
15     Three tracked configs (all re-scored every night; a clean A/B/C on the same SERVED base
knobs):
16
17     * **DEFAULT** ? the SERVED production windowing, i.e. what users get today. Catches a
regression
18     to shipped behaviour. **NOTE (post-#204, 2026-06-18):** the R1 milestone is now
flipped ON, so
19     SERVED *is* cap=6 + R4 ? "default" now equals "milestone".
20     * **MILESTONE** ? the explicit R3+R4 config (cap=6 s + rally-END inactivity trim;
21     ``docs/QUALITY_ITERATIONS.md`` R3/R4). Now == DEFAULT (production shipped it); kept
explicit so
22     the nightly still names the milestone knobs and stays meaningful if production later
diverges.
23     * **BASELINE_OFF** ? the pre-R1-flip all-OFF windowing (every quality knob forced OFF).
The
24     permanent historical floor, so each report keeps showing the delta the milestone buys
(a launched
25     win must keep standing its ground, not be taken at launch-time value).
26
27     Ground truth + trajectories are LOCAL (the manifest points at owned, minors-free
trajectory CSVs
28     outside the repo + ``output/<name>_golden_features.json``); the corpus is NOT committed.
So the
29     baseline is tied to the owner's local golden corpus and is regenerated with ``--update-
baseline``
30     whenever the corpus grows or an intended improvement lands. (A cloud agent does not
suffice ? it
31     has the repo but not the cached trajectories; hence the local Windows Scheduled Task
wiring in
32     ``scripts/nightly_regression.ps1``.)
33
34     Usage
35     -----
36     Snapshot the baseline from current HEAD (after an intended improvement or a corpus
change):
37
38     python scripts/nightly_regression.py --update-baseline
39
40     Nightly check (exit 1 on regression; writes ``output/nightly_regression/<date>.md`` +
``.json``):
41
42     python scripts/nightly_regression.py
43
44     Exit codes
45     -----
46     * ``0`` ? all tracked configs clean (no regression), corpus matches the baseline.
47     * ``1`` ? at least one regression (an aggregate metric dropped beyond tolerance, or a
per-video
48     over-seg guard increased). Takes precedence over a stale baseline.
49     * ``2`` ? operational error (manifest/corpus missing, nothing scored).
50     * ``3`` ? no baseline to compare against (snapshot one with ``--update-baseline``).
51     * ``4`` ? baseline STALE (the corpus set or a config definition changed): a refresh is

```



```

needed,
52     but no regression was found on the comparable intersection.
53     """
54     from __future__ import annotations
55
56     import argparse
57     import datetime as _dt
58     import json
59     import os
60     import sys
61     from dataclasses import dataclass, field, replace
62     from typing import Any, Dict, List, Optional, Tuple
63
64     # ----- #
65     # Paths (resolved relative to the repo root, so the script is cwd-robust).
66     # ----- #
67     REPO_ROOT = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
68     if REPO_ROOT not in sys.path:
69         sys.path.insert(0, REPO_ROOT) # importable as `python
scripts/nightly_regression.py` from anywhere
70     DEFAULT_BASELINE = os.path.join(REPO_ROOT, "eval_baselines", "nightly_baseline.json")
71     DEFAULT_REPORT_DIR = os.path.join(REPO_ROOT, "output", "nightly_regression")
72     DEFAULT_MANIFEST = os.path.join(REPO_ROOT, "output", "human_lovo_manifest.json")
73     DEFAULT_FEATURES_DIR = os.path.join(REPO_ROOT, "output")
74     DEFAULT_IOU = 0.5
75
76     # ----- #
77     # What we gate on (a tripwire ? focused, not every metric).
78     # * HIGHER-better aggregates regress when they DROP beyond `metric_tol`.
79     # * merge_rate (lower-better) regresses when it RISES beyond `rate_tol`.
80     # * per-video split_count / merge_count must NOT increase beyond `over_seg_tol`
81     # (the R2 §2.3.3 guard: "may not increase EITHER on ANY video").
82     # Boundary-error medians + seg_ratio are outlier-dominated diagnostics ? REPORTED, not
gated.
83     # ----- #
84     GATED_HIGHER: Tuple[str, ...] = ("tol_f1", "f1@0.5", "f1@0.25")
85     GATED_LOWER: Tuple[str, ...] = ("merge_rate",)
86     PER_VIDEO_GUARD: Tuple[str, ...] = ("split_count", "merge_count")
87
88     #: Aggregate metrics snapshotted into the baseline (the gated subset + report-only
context).
89     AGG_SNAPSHOT: Tuple[str, ...] = (
90         "tol_f1", "tol_precision", "tol_recall", "f1@0.5", "f1@0.25", "precision", "recall",
91         "merge_rate", "split_rate", "segment_count_ratio", "dstart_abs_median",
"dend_abs_median",
92     )
93     #: Per-video metrics snapshotted into the baseline.
94     PV_SNAPSHOT: Tuple[str, ...] = (
95         "tol_f1", "f1@0.5", "f1@0.25", "merge_rate", "split_count", "merge_count",
"num_pred", "num_gt",
96     )
97
98     DEFAULT_METRIC_TOL = 0.005 # F1-like drop that counts as a regression (absorbs float
jitter)
99     DEFAULT_RATE_TOL = 0.010 # merge_rate rise that counts as a regression
100     DEFAULT_OVER_SEG_TOL = 0 # per-video split/merge increase allowed (strict)
101
102
103     # ----- #
104     # The two tracked configs.
105     # ----- #
106     @dataclass(frozen=True)
107     class NightlyConfig:
108         """A named windowing config the nightly re-scores (preset + net-crossings gate)."""
109

```

```

110     name: str
111     preset: Any          # backend.eval.windowing.WindowingPreset
112     min_crossings: int = 0
113     note: str = ""
114
115
116     def tracked_configs() -> List[NightlyConfig]:
117         """DEFAULT (SERVED = shipped) + MILESTONE (explicit cap=6 + R4) + BASELINE_OFF (the
pre-flip floor).
118
119         All three are anchored on the SAME ``windowing.SERVED`` base knobs (velocity_thresh
/ merge_gap /
120         min_window) so they differ ONLY in the quality knobs ? they are a clean A/B/C:
121
122         * **DEFAULT** ? ``SERVED`` verbatim, so it can never silently drift from the served
operating
123         point. **Post-#204 (2026-06-18) the R1 milestone is flipped ON ? SERVED IS cap=6 +
R4, so
124         DEFAULT now equals MILESTONE.** Its job: catch an *accidental* change to a
production windowing
125         knob.
126         * **MILESTONE** ? SERVED + the explicit R3/R4 knobs (``docs/QUALITY_ITERATIONS.md``
R3/R4). Pins
127         the milestone *definition* independent of what production ships, so "is cap=6 + R4
still scoring
128         what we expect?" stays answerable even if a future change moves DEFAULT.
129         * **BASELINE_OFF** ? SERVED with every quality knob forced OFF (the pre-R1-flip
windowing). Pins
130         the *historical floor* so each nightly report keeps showing the delta the
milestone buys (the
131         dual-eval-set discipline: a launched win must keep standing its ground, not be
taken at
132         launch-time value). It is the reference DEFAULT used to be before the flip."""
133         from backend.eval.windowing import SERVED
134
135         default = NightlyConfig(
136             name="default", preset=SERVED, min_crossings=0,
137             note="SERVED production windowing ? now the R1 milestone (cap=6 + R4) after the
#204 flip.",
138         )
139         milestone = NightlyConfig(
140             name="milestone",
141             preset=replace(SERVED, name="milestone", max_window_duration=6.0,
142                           inactivity_end=1.5, inactivity_rally_thresh=0.20,
143                           inactivity_min_density=0.30),
144             min_crossings=0,
145             note="R3 cap=6 s + R4 rally-END inactivity trim ? the shipped milestone (==
default post-#204).",
146         )
147         baseline_off = NightlyConfig(
148             name="baseline_off",
149             preset=replace(SERVED, name="baseline_off", max_window_duration=0.0,
150                           inactivity_end=0.0, inactivity_rally_thresh=0.08,
151                           inactivity_min_density=0.34,

```

37. user (2026-06-22T09:09:11.697Z)

```

1     """Golden Regression Fixtures (M1) ? video-free, CONTROL-only characterization harness.
2
3     `docs/GOLDEN_REGRESSION_FIXTURES.md` ($4, $6, $7). This module is the **single shared
4     replay code path** used by both the freeze tool and the characterization gate, and it
5     deliberately stays on the **CONTROL path** (D5 in that doc): the learned
6     variants train a numpy/OpenBLAS logistic regression whose near-threshold decisions flip
7     across CPUs, so they can never anchor a cross-machine byte/parse snapshot. Here we

```

```

replay
8     only `experiment.CONTROL` (`is_learned() == False`), which keeps all candidate
intervals,
9     merges overlaps, and **avoids numpy EXECUTION on the replay path** ? i.e. no numpy array
/
10    LogisticRegression math runs while replaying. (numpy is still *imported* transitively
via
11    `experiment` ? the learned-variant classifier module and MUST be installed; the CONTROL
12    path simply never calls into it.)
13
14    The replay reuses the already-shipped eval stack VERBATIM::
15
16        parse_trajectory_csv ? distill_local.build_candidates(proposal=pinned)
17        ? VideoCandidates(name, intervals, features={5 shuttle cols},
gts=load_gt_intervals)
18        ? experiment.predict(CONTROL, vc) ? metrics.score(preds, gts, iou)
19
20    Tier-0 / privacy invariants (D3, $4c, $6 of the doc), enforced here by construction:
21
22    * Fixtures are **synthetic** ? deterministic formulae, NO randomness ? so they carry
zero
23    provenance/licensing risk and are safe to commit publicly (owner Q7 "synthetic-only").
24    * Every fixture is tagged ``kind="synthetic"`, ``minors_free=true``,
``license="synthetic"``.
25    * ``expected.json`` carries **only the 5 shuttle feature columns** ? there is, by
design, no
26    person / pose / gait / face / audio field anywhere in the schema (positive-assertion
test).
27    * The windowing params ``(vt, mg, mwd)`` are **pinned inside each fixture's provenance**
and
28    passed explicitly to ``build_candidates`` as a raw 3-tuple ? we do NOT import or rely
on the
29    named ``windowing.SERVED`` constant / ``WindowingPreset``, so a future SERVED retune
cannot
30    silently churn fixtures.
31    * ``max_win`` (W1 bounded-windowing) is **RECORDED** in provenance/manifest but NOT
applied by
32    the replay: ``distill_local.build_candidates`` (shared code, intentionally not
modified here)
33    takes only the 3-tuple ``(vt, mg, mwd)`` and
``TrackNetRunner.trajectory_to_action_windows``
34    has no max-window arg, so these fixtures characterize the **W1-OFF** path by design.
The pinned
35    ``max_win`` is a forward-looking annotation, NOT a knob the gate exercises ? do not
assume it
36    tracks a future SERVED W1 flip (it would need new wiring + a re-freeze to take
effect).
37
38    $6 honest-limits caveat (carried in the CLI / test messages, not only the docs): a green
39    suite proves only that the deterministic post-trajectory transforms are unchanged for
the
40    frozen synthetic cases ? NOT that the detector/decoder/AI handoff/DB are correct, and
NOT
41    that rally detection is good. Synthetic-tier numbers measure plumbing correctness, never
42    field quality (never cite them in QUALITY_ITERATIONS.md / the paper).
43
44    Offline + deterministic. The replay imports `tracknet_runner` (stdlib-only for the
45    parse/window math ? no TF) and reaches no numpy/cv2 EXECUTION on the CONTROL path, but
it is
46    NOT a zero-third-party-import module: importing `experiment` transitively imports numpy
(via
47    the learned-variant classifier), so numpy must be installed even though the CONTROL
replay
48    never calls into it.
49    """

```

```

50     from __future__ import annotations
51
52     import argparse
53     import json
54     import math
55     import re
56     import sys
57     from dataclasses import dataclass
58     from pathlib import Path
59     from typing import Any, Dict, List, Optional, Sequence, Tuple
60
61     # --- shipped eval stack, reused verbatim (CONTROL / pure-stdlib path only)
62     -----
63     from backend.eval import distill_local
64     from backend.eval.experiment import CONTROL, VideoCandidates, predict
65     from backend.eval.gt_loader import load_gt_intervals
66     from backend.eval.metrics import score
67     from backend.eval.regression import gt_hash
68
69     Interval = Tuple[float, float]
70
71     #: Bump when the fixture/expected schema changes shape. Loaders refuse a fixture whose
72     #: ``schema_version`` exceeds this (forward-incompatible), per the schema-drift guard.
73     CURRENT_SCHEMA = 1
74
75     #: Repo-root-relative fixtures dir, resolved from THIS file (never cwd) so the harness
76     works
77     #: from any working directory / a fresh clone. ``parents[2]`` = backend/eval ? backend ?
78     root.
79     FIXTURES_DIR: Path = Path(__file__).resolve().parents[2] / "eval_baselines" / "fixtures"
80
81     #: The pinned SERVED windowing the synthetic fixtures freeze at: (velocity_thresh,
82     merge_gap,
83     #: min_window_duration). Mirrors the historical SERVED values (0.02, 2.0, 2.0) but is a
84     LITERAL
85     #: here on purpose ? the spec forbids depending on the named ``windowing.SERVED``
86     constant so a
87     #: future retune of SERVED can never silently re-baseline a committed fixture. Each
88     fixture also
89     #: re-states these in its provenance; the replay reads them from there.
90     PINNED_VT = 0.02
91     PINNED_MERGE_GAP = 2.0
92     PINNED_MIN_WINDOW = 2.0
93     #: W1 (bounded-windowing) max-window seconds. 0 = OFF, matching the SERVED default.
94     NOTE: this is
95     #: only RECORDED (provenance/manifest) ? it is NOT applied by the replay.
96     ``_pinned_proposal``
97     #: returns just ``(vt, mg, mwd)`` and ``distill_local.build_candidates`` (shared code,
98     deliberately
99     #: NOT modified by this harness) takes no max-window arg, so the fixtures characterize
100     the W1-OFF
101     #: path. Do NOT assume this value tracks a future SERVED W1 flip ? that would need new
102     wiring.
103     PINNED_MAX_WINDOW = 0.0
104
105     #: The 5 fps/resolution-invariant shuttle feature columns produced by distill_local. Re-
106     stated
107     #: here (not imported as a mutable alias) so a fixture's expected.json is pinned to
108     exactly these.
109     SHUTTLE_FEATURES: Tuple[str, ...] = (
110         "duration", "peak_velocity", "mean_velocity", "active_ratio", "net_crossings",
111     )
112     assert tuple(distill_local.FEATURE_NAMES) == SHUTTLE_FEATURES, (
113         "distill_local.FEATURE_NAMES drifted from the pinned shuttle feature set"
114     )
115

```

38. assistant (2026-06-22T09:09:14.344Z)

Perfect! Now I have enough information. Let me create a comprehensive analysis document:

39. user (2026-06-22T09:09:15.684Z)

Chapter marked: "Determinism analysis complete". Continue your current work.

40. assistant (2026-06-22T09:09:17.861Z)

Now let me compile the structured output:

41. user (2026-06-22T09:10:12.311Z)

Structured output provided successfully

42. assistant (2026-06-22T09:10:24.297Z)

Perfect. My analysis is complete. Let me summarize the key findings:

Summary

I've completed a thorough determinism analysis of the badminton-highlight-indexer pipeline for nightly regression testing. Here are the core findings:

****Reel Count IS Deterministic ?****

The COUNT of rally segments (reels/highlights) produced by the pipeline is ****100% deterministic and safe to assert exactly**** for a fixed (video, config, weights) triple.

****Why It's Deterministic****

1. ****WASB inference**** (wasb_infer.py): Uses `torch.no\_grad()` with a deterministic detector pass. Trajectories are cached and replayed in stable frame order (sorted by frame_id).
2. ****Windowing logic**** (tracknet_runner.py): Pure Python algorithms with:
 - No RNG, seeds, or randomness
 - Sorted list operations (no dict/set iteration)
 - Strict inequality comparisons (velocity > threshold)
 - Deterministic gap-finding and midpoint selection
3. ****No nondeterminism sources****:
 - No parallelism affecting segment order
 - No time/date in counts
 - No GPU variation (trajectory cached, downstream is CPU)
 - No float threshold ties (values are frame indices / fixed fps)
 - No dict ordering issues (all sorted explicitly)
4. ****Ffmpeg note****: Keyframe placement varies across runs, but this affects BYTES not SEGMENT COUNT.

****Safe to Assert****

- ? Exact segment ****COUNT**** (list length)
- ? Segment ****boundaries**** (start/end times) to high precision if trajectory is cached
- ? Byte-identical MP4 output (ffmpeg encoder stochasticity)

****For Nightly Regression****

Assert `len(segments) == expected\_count` exactly. Use 2?3 short golden videos (30?60 sec), cache their trajectories once, snapshot expected counts, and re-score purely on windowing logic every night. This is a determinism tripwire, not a quality benchmark.